



4. The Bias-Variance Trade-Off

Overview

In this chapter, we'll cover the remaining elements of logistic regression, including what happens when you call `.fit` to train the model, and the statistical assumptions you should be aware of when using this modeling technique. You will learn how to use L1 and L2 regularization with logistic regression to prevent overfitting and how to use the practice of cross-validation to decide the regularization strength. After reading this chapter, you will be able to use logistic regression in your work and employ regularization in the model fitting process to take advantage of the bias-variance trade-off and improve model performance on unseen data.

Introduction

In this chapter, we will introduce the remaining details of logistic regression left over from the previous chapter. In addition to being able to use scikit-learn to fit logistic regression models, you will gain insight into the gradient descent procedure, which is similar to the processes that are used "under the hood" (invisible to the user) to accomplish model fitting in scikit-learn. Finally, we'll complete our discussion of the logistic regression model by familiarizing ourselves with the formal statistical assumptions of this method.

We begin our exploration of the foundational machine learning concepts of overfitting, underfitting, and the bias-variance trade-off by examining how the logistic regression model can be extended to address the overfitting problem. After reviewing the mathematical details of the regularization methods that are used to alleviate overfitting, you will learn a useful

practice for tuning the hyperparameters of regularization: cross-validation. Through the methods of regularization and some simple feature engineering, you will gain an understanding of how to improve both overfitted and underfitted models.

Although we are focusing on logistic regression in this chapter, the concepts of overfitting, underfitting, regularization, and the bias-variance trade-off are relevant to nearly all supervised modeling techniques in machine learning.

Estimating the Coefficients and Intercepts of Logistic Regression

In the previous chapter, we learned that the coefficients of a logistic regression model (each of which goes with a particular feature), as well as the intercept, are determined using the training data when the `.fit` method is called on a logistic regression model in scikit-learn. These numbers are called the **parameters** of the model, and the process of finding the best values for them is called parameter **estimation**. Once the parameters are found, the logistic regression model is essentially a finished product: with just these numbers, we can use a logistic regression model in any environment where we can perform common mathematical functions.

It is clear that the process of parameter estimation is important, since this is how we can make a predictive model from our data. So, how does parameter estimation work? To understand this, the first step is to familiarize ourselves with the concept of a **cost function**. A cost function is a way of telling how far away the model predictions are from perfectly describing the data. The larger the difference between the model predictions and the actual data, then the larger the "cost" returned by the cost function.

This is a straightforward concept for regression problems: the difference between predictions and true values can be used for the cost, after going through a transformation (such as absolute value or squaring) to make

the value of the cost positive, and then averaging this over all the training samples.

For classification problems, especially in fitting logistic regression models, a typical cost function is the **log-loss** function, also called cross-entropy loss. This is the cost function that scikit-learn uses, in a modified form, to fit logistic regression:

$$\log \text{loss} = \frac{1}{n} \sum_{i=1}^n - (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

Figure 4.1: The log-loss function

Here, there are n training samples, y_i is the true label (0 or 1) of the i^{th} sample, p_i is the predicted probability that the label of the i^{th} sample equals 1, and $\log$ is the natural logarithm. The summation notation (that is, the uppercase Greek letter, sigma) over all the training samples and division by n serve to take the average of this cost function over all training samples. With this in mind, take a look at the following graph of the natural logarithm function and consider what the interpretation of this cost function is:

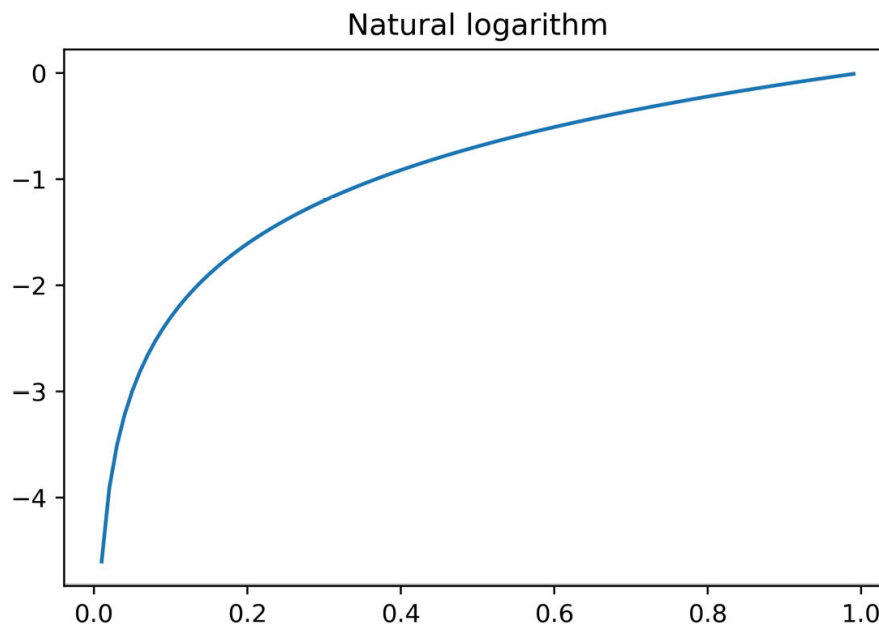


Figure 4.2: Natural logarithm on the interval (0, 1)

To see how the log-loss cost function works, consider its value for a sample where the true label is 1, which is $y = 1$ in this case, so the second part

of the cost function, $(1 - y_i)\log(1 - p_i)$, will be exactly equal to 0 and will not affect the value. Then the value of the cost function is $-y_i\log(p_i) = -\log(p_i)$ since $y_i = 1$. So, the cost for this sample is simply the negative of the natural logarithm of the predicted probability. Now since the true label for the sample is 1, consider how the cost function should behave. We expect that for predicted probabilities that are close to 1, the cost function will be small, representing a small error for predictions that are closer to the true value. For predictions that are closer to 0, it will be larger, since the cost function is supposed to take on larger values the more "wrong" the prediction is.

From the graph of the natural logarithm in *Figure 4.2* we can see that for values of p that are closer to 0, the natural logarithm takes on increasingly negative values. This means the cost function will take on increasingly positive values, so that the cost of classifying a positive sample with a very low probability is relatively high, as it should be. Conversely, if the predicted probability is closer to 1, then the graph indicates the cost will be closer to 0 – again, this is as expected for a prediction that is "more correct." Therefore, the cost function behaves as expected for a positive sample. A similar observation can be made for samples where the true label is 0.

Now we understand how the log-loss cost function works for logistic regression. But what does this have to do with how the coefficients and the intercept are determined? We will learn in the next section.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/NeF8P>.

Gradient Descent to Find Optimal Parameter Values

The problem of finding the parameter values (coefficients and intercept) for a logistic regression model using a log-loss cost boils down to a problem of **optimization**: we would like to find the set of parameters that results in the **minimum** cost, since costs are higher for worse predictions.

In other words, we want the set of parameters that is the "least wrong" on average over all of the training samples. This process is done for you automatically by the `.fit` method of the logistic regression model in scikit-learn. There are different solution techniques for finding the set of parameters with the lowest cost, and you can choose which one you would like to use with the `solver` keyword when you are instantiating the model class. All of these methods work somewhat differently. However, they are all based on the concept of **gradient descent**.

The gradient descent process starts with an **initial guess**. The choice of the initial guess is not that important for logistic regression and you don't need to make it manually; this is handled by the `solver` keyword.

However, for more advanced machine learning algorithms such as deep neural networks, selection of the initial guesses for parameters requires more attention.

For the sake of illustration, we will consider a problem where there is only one parameter to estimate. We'll look at the value of a hypothetical cost function ($y = f(x) = x^2 - 2x$) and devise a gradient descent procedure to find the value of the parameter, x , for which the cost, y , is the lowest. Here, we choose some x values, create a function that returns the value of the cost function, and look at the value of the cost function over this range of parameters.

The code to do this is as follows:

```
X_poly = np.linspace(-3,5,81)
print(X_poly[:5], '...', X_poly[-5:])
```

Here is the output of the print statement:

```
[-3. -2.9 -2.8 -2.7 -2.6] ... [4.6 4.7 4.8 4.9 5. ]
```

The remaining code snippet is as follows:

```
def cost_function(X):
    return X * (X-2)

y_poly = cost_function(X_poly)
plt.plot(X_poly, y_poly)
```

```
plt.xlabel('Parameter value')
plt.ylabel('Cost function')
plt.title('Error surface')
```

The resulting plot should appear as follows:

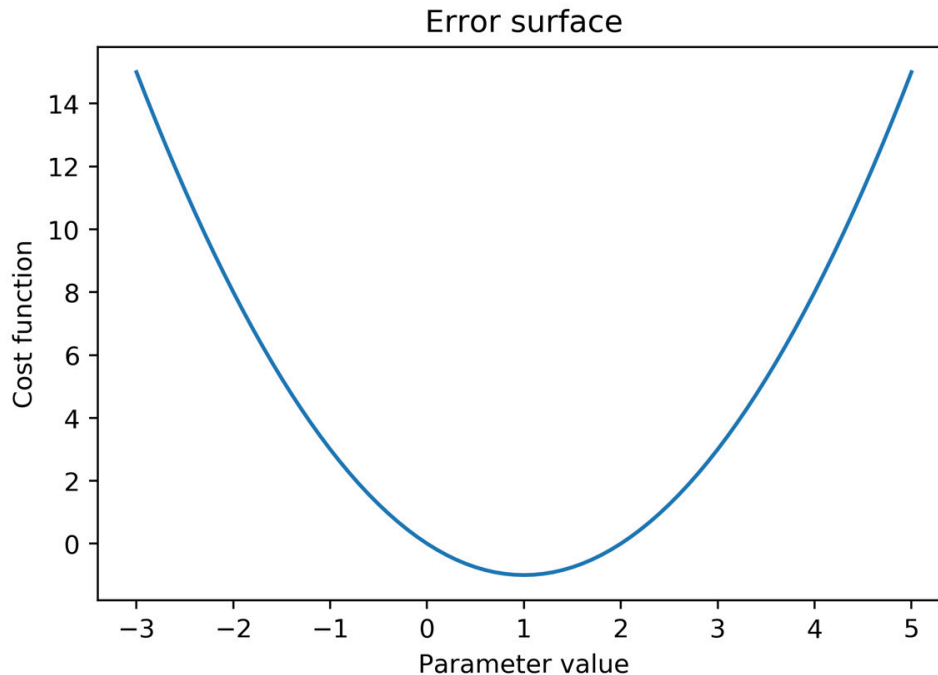


Figure 4.3: A cost function plot

Note

In the preceding code snippets, we assume that you would have imported the necessary libraries. You can refer to the following notebook for the complete code for the chapter including the import statement for the preceding snippets: <https://packt.link/A4VyF>.

Looking at the **error surface** in Figure 4.3, which is the plot of the cost function over a range of parameter values, it's pretty evident what parameter value will result in the lowest value of the cost function: $x = 1$. In fact, with some calculus, you could easily confirm this by setting the derivative to zero and then solving for x , confirming that $x = 1$ is the minimum. However, generally speaking, it is not always feasible to solve the problem so simply. In cases where it is necessary to use gradient descent, we don't always know how the entire error surface looks. Rather, after we've chosen the initial guess for the parameter, all we're able to know is the direction of the error surface in the immediate vicinity of that point.

Gradient descent is an iterative algorithm; starting from the initial guess, we try to find a new guess that lowers the cost function and continue with this until we've found a good solution. We are trying to move "downhill" on the error surface, but we only know which direction to move in and how far to move in that direction, based on the shape of the error surface in the immediate neighborhood of our current guess. In mathematical terms, we only know the value of the **derivative** (which is called the **gradient** in more than one dimension) at the parameter value of the current guess. If you have not studied calculus, you can think of the gradient as telling you which direction is downhill, and how steep the hill is from where you're standing. We use this information to "take a step" in the direction of decreasing error. How big a step we decide to take depends on the **learning rate**. Since the gradient declines toward the direction of decreasing error, we want to take a step in the direction that is the negative of the gradient.

These notions can be formalized in the following equation. To get to the new guess, x_{new} , from the current guess, x_{old} , where $f'(x_{\text{old}})$ is the derivative (that is, the gradient) of the cost function at the current guess:

$$x_{\text{new}} = x_{\text{old}} - f'(x_{\text{old}}) \times \text{learning rate}$$

Figure 4.4: Equation to obtain the new guess from the current guess

In the following graph, we can see the results of starting a gradient descent procedure from $x = 4.5$, with a learning rate of 0.75, and then optimizing x to attain the lowest value of the cost function:

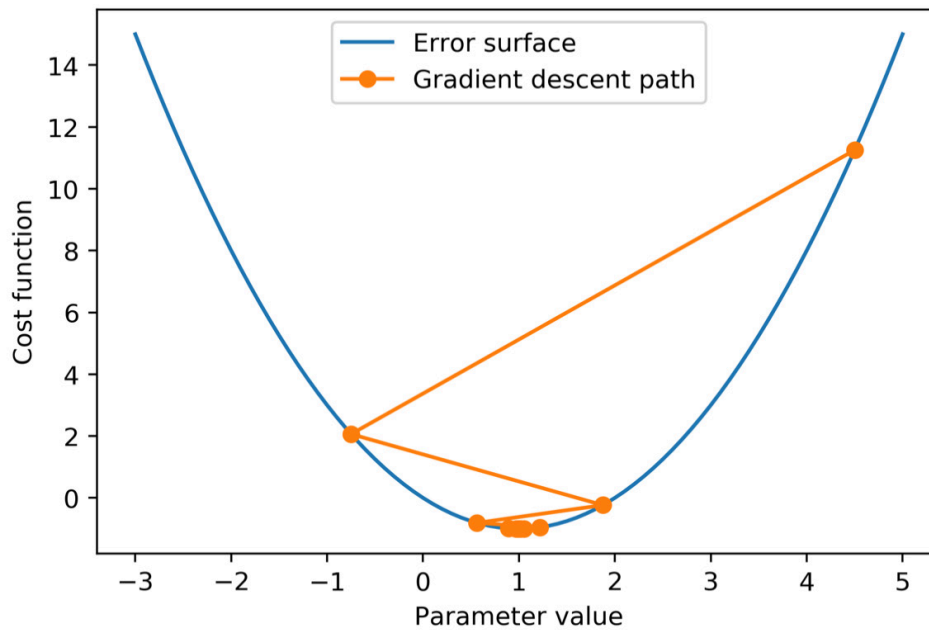


Figure 4.5: The gradient descent path

Gradient descent also works in higher-dimensional spaces; in other words, with more than one parameter. However, you can only visualize up to a two-dimensional error surface (that is, two parameters at a time on a three-dimensional plot) on a single graph.

Having described the workings of gradient descent, let's perform an exercise to implement the gradient descent algorithm, expanding on the example of this section.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/NeF8P>. If you're reading the print version of this book, you can download and browse the color versions of some of the images in this chapter by visiting the following link:

<https://packt.link/FAXBM>

Exercise 4.01: Using Gradient Descent to Minimize a Cost Function

In this exercise, our task is to find the best set of parameters in order to minimize the following hypothetical cost function: $y = f(x) = x^2 - 2x$. To do

this, we will employ gradient descent, which was described in the preceding section. Perform the following steps to complete the exercise:

Note

Before you begin this exercise, please make sure you have executed the prerequisite steps of importing the necessary libraries and loading the cleaned dataframe. These steps along with the code for this exercise can be found at <https://packt.link/NeF8P>.

1. Create a function that returns the value of the cost function and look at the value of the cost function over a range of parameters. You can use the following code to do this (note that this repeats code from the preceding section):

```
X_poly = np.linspace(-3,5,81)
print(X_poly[:5], '...', X_poly[-5:])
def cost_function(X):
    return X * (X-2)
y_poly = cost_function(X_poly)
plt.plot(X_poly, y_poly)
plt.xlabel('Parameter value')
plt.ylabel('Cost function')
plt.title('Error surface')
```

You will obtain the following plot of the cost function:

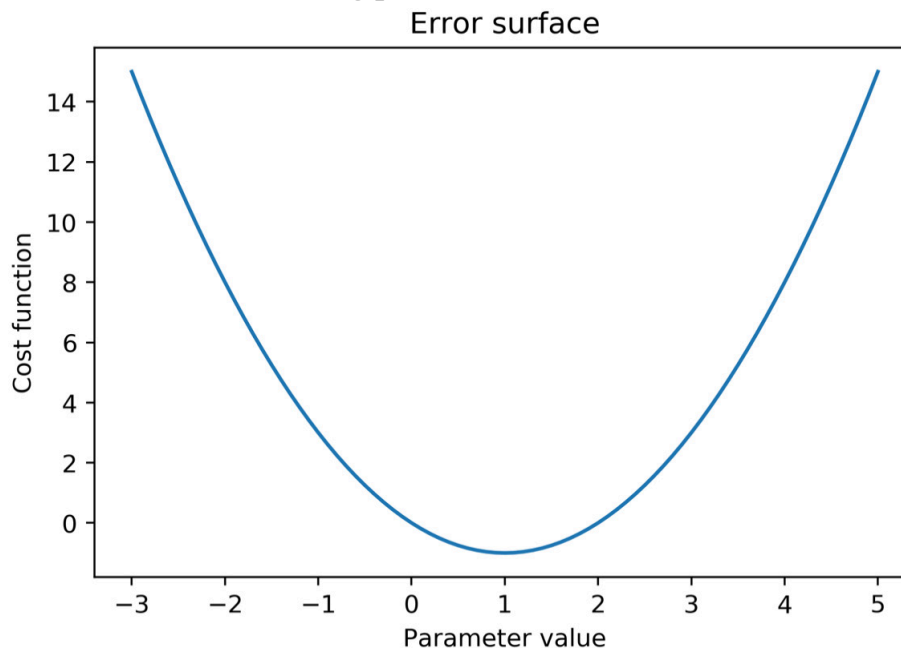


Figure 4.6: A cost function plot

2. Create a function for the value of the gradient. This is the analytical derivative of the cost function. Use this function to evaluate the gradient at the point $x = 4.5$, and then use this in combination with the learning rate to find the next step of the gradient descent process:

```
def gradient(X):
    return (2*X) - 2
x_start = 4.5
learning_rate = 0.75
x_next = x_start - gradient(x_start)*learning_rate
x_next
```

Note

It doesn't matter if you haven't studied calculus and don't understand this part; you can just take it as a given that this is the function for the gradient. In some applications, it's not actually possible to calculate an analytical derivative, so this may need to be numerically approximated.

After running the cell with `x_next`, you will obtain the following output:

```
-0.75
```

This is the next gradient descent step after $x = 4.5$.

3. Plot the gradient descent path, from the starting point to the next point, using the following code:

```
plt.plot(X_poly, y_poly)
plt.plot([x_start, x_next],
         [cost_function(x_start),
          cost_function(x_next)],
         '-o')
plt.xlabel('Parameter value')
plt.ylabel('Cost function')
plt.legend(['Error surface', 'Gradient descent path'])
```

You will obtain the following output:

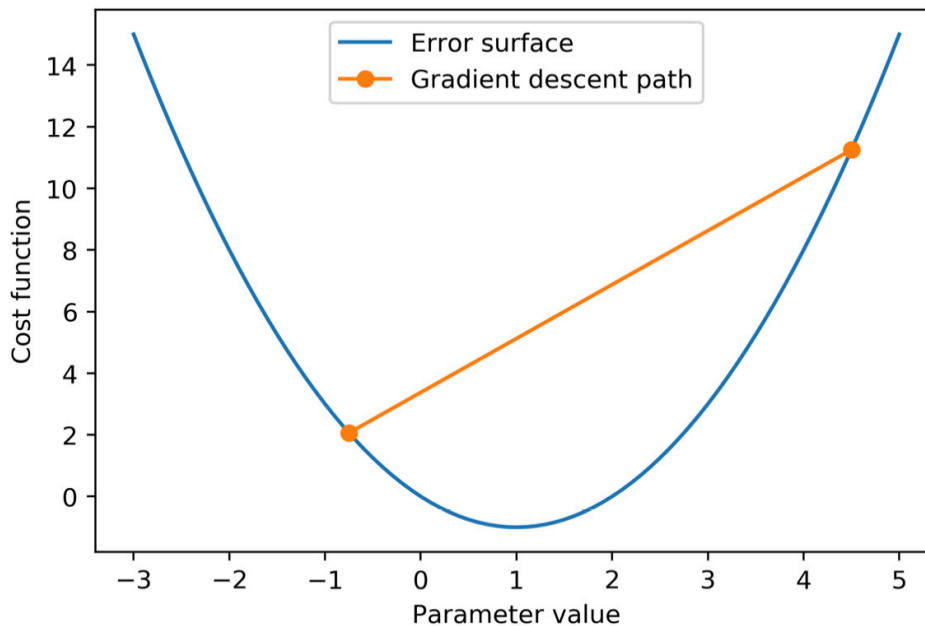


Figure 4.7: The first gradient descent path step

Here, it appears as though we've taken a step in the right direction. However, it's clear that we've overshoot where we want to be. It may be that our learning rate is too large, and consequently, we are taking steps that are too big. While tuning the learning rate will be a good way to converge toward an optimal solution more quickly, in this example, we can just continue illustrating the remainder of the process. Here, it looks like we may need to take a few more steps. In practice, gradient descent continues until the size of the steps become very small, or the change in the cost function becomes very small (you can specify how small by using the `tol` argument in the scikit-learn logistic regression), indicating that we're close enough to a good solution – that is, a **local minimum** of the cost function. For this example, we'll just take a total of 14 steps, or **iterations**, beyond the initial guess (note that you can also set the maximum number of iterations in scikit-learn with `max_iter`).

4. Perform 14 iterations to converge toward the local minimum of the cost function by using the following code snippet (note that `iterations = 15`, but the endpoint is not included in the call to `range()`):

```
iterations = 15
x_path = np.empty(iterations,)
x_path[0] = x_start
for iteration_count in range(1, iterations):
    derivative = gradient(x_path[iteration_count-1])
```

```

        x_path[iteration_count] = x_path[iteration_count-1]
        \
        -
        (derivative*learning_rate)
    x_path

```

You will obtain the following output:

```

array([ 4.5 , -0.75 , 1.875 , 0.5625 , 1.21875 ,
        0.890625 , 1.0546875 , 0.97265625, 1.01367188,
        0.99316406,
        1.00341797, 0.99829102, 1.00085449, 0.99957275,
        1.00021362])

```

This **for** loop stores the successive estimates in the **x_path** array, using the current estimate to calculate the derivative and find the next estimate. From the resulting values of the gradient descent process, it looks like we've gotten very close (**1.00021362**) to the optimal solution of 1.

5. Plot the gradient descent path using the following code:

```

plt.plot(X_poly, y_poly)
plt.plot(x_path, cost_function(x_path), '-o')
plt.xlabel('Parameter value')
plt.ylabel('Cost function')
plt.legend(['Error surface', 'Gradient descent path'])

```

You will obtain the following output:

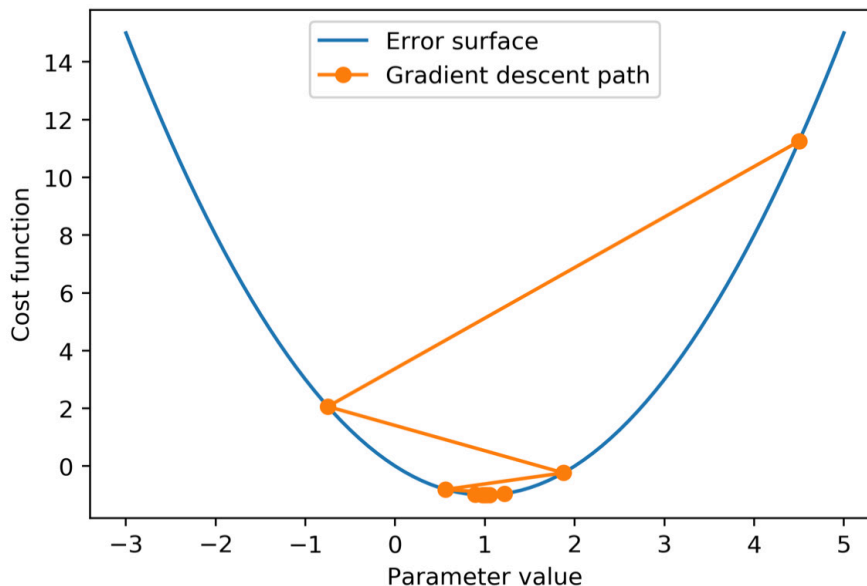


Figure 4.8: The gradient descent path

We encourage you to repeat the previous procedure with different learning rates in order to see how they affect the gradient descent path. With the right learning rate, it's possible to converge on a highly accurate solution very quickly. While the choice of learning rate can be important in different machine learning applications, for logistic regression, the problem is usually pretty easy to solve and you don't need to select a learning rate in scikit-learn.

As you experimented with different learning rates, did you notice what happened when the learning rate was greater than one? In this case, the step that we take in the direction of the decreasing error is too large and we actually wind up with a higher error. This problem can compound itself and actually lead the gradient descent process away from the region of minimum error. On the other hand, if the step size is too small, it can take a very long time to find the desired solution.

Assumptions of Logistic Regression

Since it is a classical statistical model, similar to the F-test and Pearson correlation we already examined, logistic regression makes certain assumptions about the data. While it's not necessary to follow every one of these assumptions in the strictest possible sense, it's good to be aware of them. That way, if a logistic regression model is not performing very well, you can try to investigate and figure out why, using your knowledge of the ideal situation that logistic regression is intended for. You may find slightly different lists of the specific assumptions from different resources. However, those that are listed here are widely accepted.

Features Are Linear in the Log Odds

We learned about this assumption in the previous chapter, *Chapter 3, Details of Logistic Regression and Feature Exploration*. Logistic regression is a linear model, so it will only work well as long as the features are effective at describing a linear trend in the log odds. In particular, logistic regression won't capture interactions, polynomial features, or the discretization of features, on its own. You can, however, specify all of these

as "new features" – even though they may be engineered from existing features.

Remember from the previous chapter that the most important feature from univariate feature exploration, **PAY_1**, was not found to be linear in the log odds.

No Multicollinearity of Features

Multicollinearity means that features are correlated with each other. The worst violation of this assumption is when features are perfectly correlated with each other, such as one feature being identical to another, or when one feature equals another multiplied by a constant. We can investigate the correlation of features using the correlation plot that we're already familiar with from univariate feature selection. Here is the correlation plot from the previous chapter:

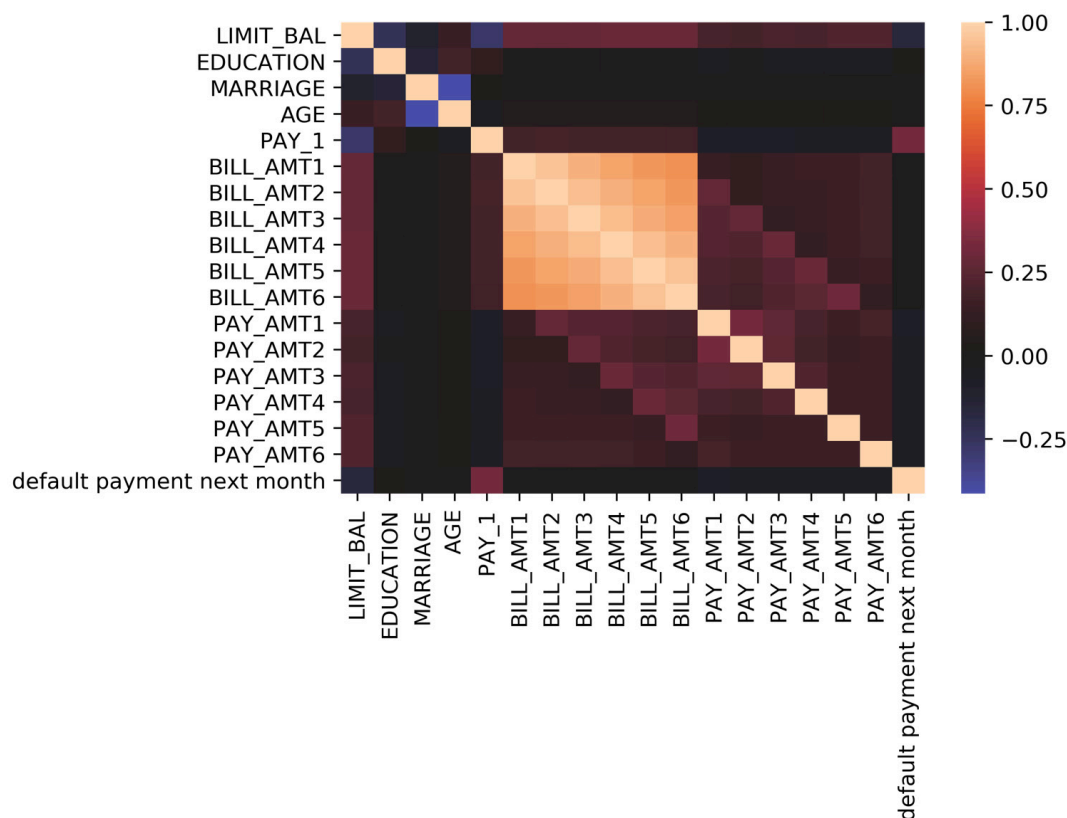


Figure 4.9: A correlation plot of features and the response

We can see from the correlation plot what perfect correlation looks like: since every feature and the response variable has a correlation of 1 with

itself, we can see that a correlation of 1 is a light, cream color. From the color bar, which doesn't include -1, we know there are no correlations with that value.

Note

The Jupyter notebook containing the code and the corresponding plots presented in this section can be found here: <https://packt.link/UOEMp>.

The clearest examples of correlated predictors in our case study data are the **BILL_AMT** features. It makes intuitive sense that bills might be similar from month to month for a given account. For instance, there may be an account that typically carries a balance of zero, or an account that has a large balance that is taking a while to pay off. Are any of the **BILL_AMT** features perfectly correlated? From *Figure 4.9*, it does not look like it. So, while these features may not contribute much independent information, we won't remove them at this point out of concern for multicollinearity.

The Independence of Observations

This is a common assumption in classical statistical models, including linear regression. Here, the observations (or samples) are assumed to be independent. Does this make sense with the case study data? We'd want to confirm with our client whether the same individual can hold multiple credit accounts across the dataset and consider what to do depending on how common it was. Let's assume we've been told that in our data each credit account belongs to a unique person, so we may assume independence of observations in this respect.

Across different domains of data, some common violations of independence of observations are as follows:

- **Spatial autocorrelation** of observations; for example, in natural phenomena such as soil types, where observations that are geographically close to each other may be similar to each other.
- **Temporal autocorrelation** of observations, which may occur in time series data. Observations at the current point in time are usually as-

sumed to be correlated to the most recent point(s) in time.

However, these issues are not relevant to our case study data.

No Outliers

Outliers are observations where the value of the feature(s) or response are very far from most of the data or are different in some other way. A more appropriate term for an outlier observation of a feature value is a high leverage point, as the term "outlier" is usually applied to the response variable. However, in our binary classification problem, it's not possible to have an outlier value of the response variable, since it can only take on the values 0 and 1. In practice, you may see both of these terms used to refer to features.

To see why these kinds of points can have an adverse effect on linear models in general, take a look at this synthetic linear data with 100 points and the line of best fit that results from linear regression:

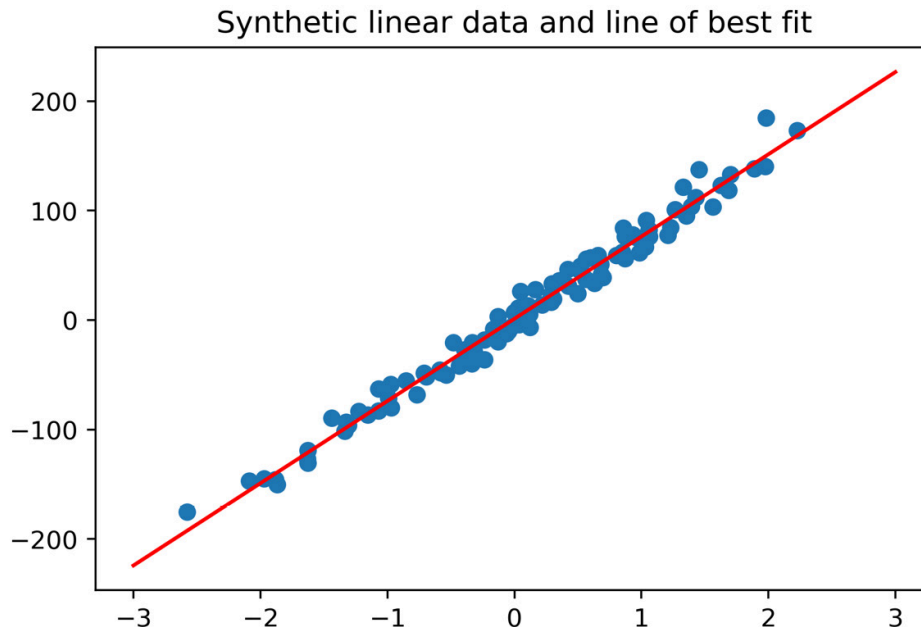


Figure 4.10: "Well-behaved" linear data and a regression fit

Here, the model intuitively appears to be a good fit for the data. However, what if an outlier feature value is added? To illustrate this, we add a point with an x value that is very different from most of the observations and a

y value that is in a similar range to the other observations. We then show the resulting regression line:

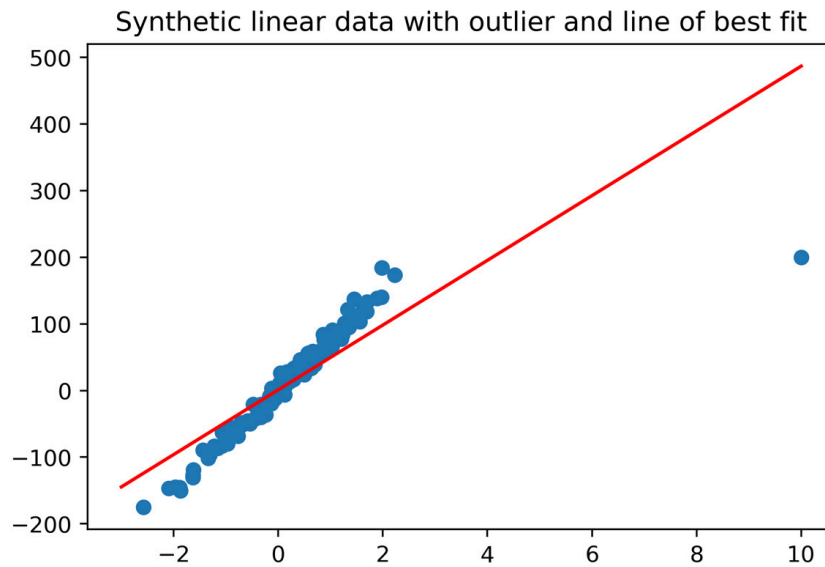


Figure 4.11: A plot showing what happens when an outlier is included

Due to the presence of a single high leverage point, the regression model fit for all the data is no longer a very good representation of much of the data. This shows the potential effect of just a single data point on linear models, especially if that point doesn't appear to follow the same trend as the rest of the data.

There are methods for dealing with outliers. But a more fundamental question to ask is "Is data like this realistic?". If the data doesn't seem right, it is a good idea to ask the client whether the outliers are believable. If not, they should be excluded. However, if they do represent valid data, then non-linear models or other methods should be used.

With our case study data, we did not observe outliers in the histograms that we plotted during feature exploration. Therefore, we don't have this concern.

How Many Features Should You Include?

This is not so much an assumption as it is guidance on model building. There is no clear-cut law that states how many features to include in a logistic regression model. However, a common rule of thumb is the "rule of

10," which states that for every 10 occurrences of the rarest outcome class, 1 feature may be added to the model. So, for example, in a binary logistic regression problem with 100 samples, if the class balance has 20% positive outcomes and 80% negative outcomes, then there are only 20 positive outcomes in total, and so only 2 features should be used in the model. A "rule of 20" has also been suggested, which would be a more stringent limit on the number of features to include (1 feature in our example).

Another point to consider in the case of binary features, such as those that result from one-hot encoding, is how many samples will have a positive value for that feature. If the feature is very imbalanced, in other words, with very few samples containing either a 1 or a 0, it may not make sense to include it in the model.

For the case study data, we are fortunate to have a relatively large number of samples and relatively balanced features, so these are not concerns.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/SnX3y>.

The Motivation for Regularization: The Bias-Variance Trade-Off

We can extend the basic logistic regression model that we have learned about by using a powerful concept known as **shrinkage** or **regularization**. In fact, every logistic regression that you have fit so far in scikit-learn has used some amount of regularization. That is because it is a default option in the logistic regression model object. However, until now, we have ignored it.

As you learn about these concepts in greater depth, you will also become familiar with a few foundational concepts in machine learning: **overfitting**, **underfitting**, and the **bias-variance trade-off**. A model is said to

overfit the training data if the performance of the model on the training data (for example, the ROC AUC) is substantially better than the performance on a held-out test set. In other words, good performance on the training set does not generalize to the unseen test set. We started to discuss these concepts in *Chapter 2, Introduction to Scikit-Learn and Model Evaluation*, when we distinguished between model training and test scores.

When a model is overfitted to the training data, it is said to have high **variance**. In other words, whatever variability exists in the training data, the model has learned this very well – in fact, too well. This will be reflected in a high model training score. However, when such a model is used to make predictions on new and unseen data, the performance is lower. Overfitting is more likely in the following circumstances:

- There are a large number of features available in relation to the number of samples. In particular, there may be so many possible features that it is cumbersome to directly inspect all of them, like we were able to do with the case study data.
- A complex model, that is, more complex than logistic regression, is used. These include models such as gradient boosting ensembles or neural networks.

Under these circumstances, the model has an opportunity develop more complex **hypotheses** about the relationships between features and the response variable in the training data during model fitting, making overfitting more likely.

In contrast, if a model is not fitting the training data very well, this is known as underfitting, and the model is said to have high **bias**.

We can examine the differences between underfitting, overfitting, and the ideal that sits in between, by fitting polynomial models on some hypothetical data:

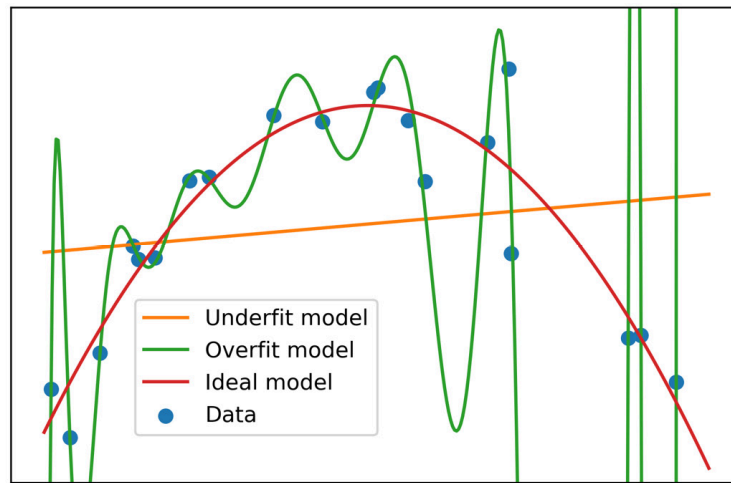


Figure 4.12: Quadratic data with underfit, overfit, and ideal models

In *Figure 4.12*, we can see that including too few features, in this case, a linear model of y with just two features, a slope and an intercept, is clearly not a good representation of the data. This is known as an underfit model. However, if we include too many features, that is, many high-degree polynomial terms, such as x^2 , x^3 , x^4 , ... x^{10} , we can fit the training data almost perfectly. However, this is not necessarily a good thing. When we look at the results of the overfitted model in between the training data points, where new predictions may need to be made, we can see that the model is unstable and may not provide reliable predictions for data that was not in the training set. We can tell this just based on an intuitive understanding of the relationship between the features and the response variable, which we can get from visualizing the data.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/SnX3y>.

The synthetic data for this example was generated by a second-degree (that is, quadratic) polynomial. Knowing this, we could easily find the ideal model by fitting a second-degree polynomial to the training data, as shown in *Figure 4.12*.

In general, however, we won't know what the ideal model formulation is ahead of time. For this reason, we need to compare training and test scores to assess whether a model may be overfitting or underfitting.

In some cases, it may be desirable to introduce some bias into the model training process, especially if this decreases overfitting and increases model performance on new, unseen data. In this way, it may be possible to leverage the bias-variance trade-off to improve a model. We can use **regularization** methods to accomplish this. Additionally, we may also be able to use these methods for **variable selection** as part of the modeling process. Using a predictive model to select variables is an alternative to the univariate feature selection methods that we've already explored. We begin to experiment with these concepts in the following exercise.

Exercise 4.02: Generating and Modeling Synthetic Classification Data

In this exercise, we'll observe overfitting in practice by using a synthetic dataset. Consider yourself in the situation of having been given a binary classification dataset with many candidate features (200), where you don't have time to look through all of them individually. It's possible that some of these features are highly correlated or related in some other way. However, with this many variables, it can be difficult to effectively explore all of them. Additionally, the dataset has relatively few samples: only 1,000. We are going to generate this challenging dataset by using a feature of scikit-learn that allows you to create synthetic datasets for making conceptual explorations such as this. Perform the following steps to complete the exercise:

Note

Before you begin this exercise, please make sure you have executed the prerequisite steps of importing the necessary libraries. These steps along with the code for this exercise can be found at <https://packt.link/mIMsT>.

1. Import the `make_classification`, `train_test_split`, `LogisticRegression`, and `roc_auc_score` classes using the following code:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
```

Notice that we've imported several familiar classes from scikit-learn, in addition to a new one that we haven't seen before:

make_classification. This class does just what its name indicates – it makes data for a classification problem. Using the various keyword arguments, you can specify how many samples and features to include, and how many classes the response variable will have. There is also a range of other options that effectively control how "easy" the problem will be to solve.

Note

For more information, refer to https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_classification.html.

Suffice to say that we've selected options here that make a reasonably easy-to-solve problem, with some curveballs thrown in. In other words, we expect high model performance, but we'll have to work a little bit to get it.

2. Generate a dataset with two variables, **x_synthetic** and **y_synthetic**. **x_synthetic** has the 200 candidate features, and **y_synthetic** the response variable, each for 1,000 samples. Use the following code:

```
X_synthetic, y_synthetic = make_classification(
    n_samples=1000, n_features=200,
    n_informative=3, n_redundant=10,
    n_repeated=0, n_classes=2,
    n_clusters_per_class=2,
    weights=None, flip_y=0.01,
    class_sep=0.8, hypercube=True,
    shift=0.0, scale=1.0,
    shuffle=True, random_state=24)
```

3. Examine the shape of the dataset and the class fraction of the response variable using the following code:

```
print(X_synthetic.shape, y_synthetic.shape)
print(np.mean(y_synthetic))
```

You will obtain the following output:

```
(1000, 200) (1000,)
0.501
```

After checking the shape of the output, note that we've generated an almost perfectly balanced dataset: close to a 50/50 class balance. It is also important to note that we've generated all the features so that they have the same **shift** and **scale** – that is, a mean of 0 with a standard deviation of 1. Making sure that the features are on the same scale, or have roughly the same range of values, is a key point for using regularization methods – and we'll see why later. If the features in a raw dataset are on widely different scales, it is advisable to normalize them so that they are on the same scale. Scikit-learn has the functionality to make this easy, which we'll learn about in the activity at the end of this chapter.

4. Plot the first few features as histograms to show that the range of values is the same using the following code:

```
for plot_index in range(4):
    plt.subplot(2, 2, plot_index+1)
    plt.hist(X_synthetic[:, plot_index])
    plt.title('Histogram for feature
{}'.format(plot_index+1))
plt.tight_layout()
```

You will obtain the following output:

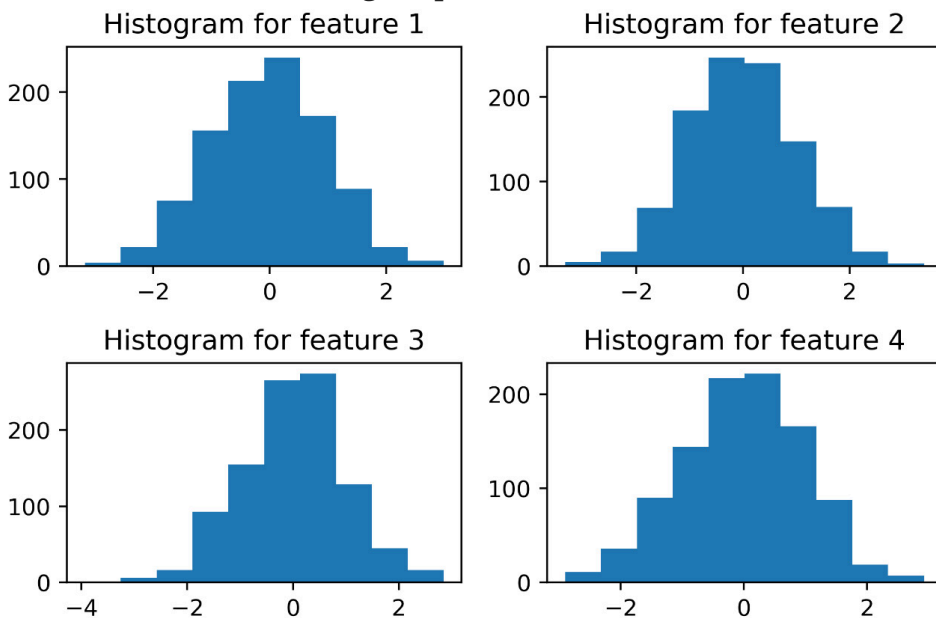


Figure 4.13: Histograms for the first 4 of 200 synthetic features

Because we generated this dataset, we don't need to directly examine all 200 features to make sure that they're on the same scale. So, what are the possible concerns with this dataset? The data is balanced in

terms of the class fractions of the response variable, so we don't need to undersample, oversample, or use other methods that are helpful for imbalanced data. What about relationships among the features themselves, and the features and response variable? There are a lot of these relationships and it is a challenge to investigate them all directly.

Based on our rule of thumb (that is, 1 feature allowed for every 10 samples of the rarest class), 200 features is too many. We have 500 observations in the rarest class, so by that rule, we shouldn't have more than 50 features. It's possible that with so many features, the model training procedure will overfit. We will now start to learn how to use options in the scikit-learn logistic regression to prevent this.

5. Split the data into training and test sets using an 80/20 split, and then instantiate a logistic regression model object using the following code:

```
X_syn_train, X_syn_test, y_syn_train, y_syn_test = \
train_test_split(X_synthetic, y_synthetic, \
                  test_size=0.2, random_state=24)
lr_syn = LogisticRegression(solver='liblinear',
                             penalty='l1',
                             C=1000, random_state=1)
lr_syn.fit(X_syn_train, y_syn_train)
```

Notice here that we are specifying some new options in the logistic regression model, which, so far, we have not paid attention to. First, we specify the **penalty** argument to be **l1**. This means we are going to use **L1 regularization**, which is also known as **lasso regularization**. We'll discuss the mathematical definition of this shortly. Second, notice that we have set the **C** parameter to be equal to 1,000. **C** is the "inverse of regularization strength," according to the scikit-learn documentation

(https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html).

This means that higher values of **C** correspond to less regularization.

By choosing a relatively large number, such as 1,000, we are using relatively little regularization. The default value of **C** is 1. So, we are not really using much regularization here, rather, we are simply becoming familiar with the options to do so. Finally, we are using the **lib-linear** solver, which we have used in the past.

Although we happen to be using scaled data here (all features have a mean of 0 and standard deviation of 1), it's worth noting at this point that among the various options we have available for solvers, **liblinear** is "robust to unscaled data." Also note that **liblinear** is one of only two solver options that support the L1 penalty – the other option being **saga**.

Note

You can find out more information on available solvers at https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

6. Fit the logistic regression model on the training data using the following code:

```
lr_syn.fit(X_syn_train, y_syn_train)
```

Here is the output:

```
LogisticRegression(C=1000, penalty='l1',
random_state=1, \
                    solver='liblinear')
```

7. Calculate the training score using this code by first getting predicted probabilities and then obtaining the ROC AUC:

```
y_syn_train_predict_proba =
lr_syn.predict_proba(X_syn_train)
roc_auc_score(y_syn_train,
y_syn_train_predict_proba[:,1])
```

The output should be as follows:

```
0.9420000000000001
```

8. Calculate the test score similar to how the training score was obtained:

```
y_syn_test_predict_proba =
lr_syn.predict_proba(X_syn_test)
roc_auc_score(y_syn_test,
y_syn_test_predict_proba[:,1])
```

The output should be as follows:

```
0.8075807580758075
```

From these results, it's apparent that the logistic regression model has overfit the data. That is, the ROC AUC score on the training data is substantially higher than that of the test data.

Lasso (L1) and Ridge (L2) Regularization

Before applying regularization to a logistic regression model, let's take a moment to understand what regularization is and how it works. The two ways of regularizing logistic regression models in scikit-learn are called **lasso** (also known as **L1** regularization) and **ridge** (also known as **L2** regularization). When instantiating the model object from the scikit-learn class, you can choose `penalty = 'l1'` or `'l2'`. These are called "penalties" because the effect of regularization is to add a penalty, or a cost, for having larger values of the coefficients in a fitted logistic regression model.

As we've already learned, coefficients in a logistic regression model describe the relationship between the log odds of the response and each of the features. Therefore, if a coefficient value is particularly large, then a small change in that feature will have a large effect on the prediction. When a model is being fit and is learning the relationship between features and the response variable, the model can start to learn the noise in the data. We saw this previously in *Figure 4.12*: if there are many features available when fitting a model, and there are no guardrails on the values that their coefficients can take, then the model fitting process may try to discover relationships between the features and the response variable that won't generalize to new data. In this way, the model becomes tuned to the unpredictable, random noise that accompanies real-world, imperfect data. Unfortunately, this only serves to increase the model's skill at predicting the training data, which is not our ultimate goal. Therefore, we should seek to root out such spurious relationships from the model.

Lasso and ridge regularization use different mathematical formulations to accomplish this goal. These methods work by making changes to the cost function that is used for model fitting, which we introduced previously as the log-loss function. Lasso regularization uses what is called the **1-norm** (hence the term L1):

$$\log \text{ loss with lasso penalty} = \sum_{j=1}^m |\sigma_j| + \frac{c}{n} \sum_{i=1}^n - (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

Figure 4.14: Log-loss equation with lasso penalty

The 1-norm, which is the first term in the equation in *Figure 4.14*, is just the sum of the absolute values of the coefficients of the m different features. The absolute value is used because having a coefficient that's large in either the positive or negative directions can contribute to overfitting. So, what else is different about this cost function compared to the log-loss function that we saw earlier? Well, now there is a C factor that is multiplied by the fraction in front of the sum of the log-loss function.

This is the "inverse of regularization strength," as described in the scikit-learn documentation (https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html).

Since this factor is in front of the term of the cost function that calculates the prediction error, as opposed to the term that does regularization, then making it larger makes the prediction error more important in the cost function, while regularization is made less important. In short, *larger values of C lead to less regularization* in the scikit-learn implementation.

L2, or ridge regularization, is similar to L1, except that instead of the sum of absolute values of coefficients, ridge uses the sum of their squares, called the **2-norm**:

$$\text{log loss with ridge penalty} = \sum_{j=1}^m \sigma_j^2 + \frac{c}{n} \sum_{i=1}^n -(y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

Figure 4.15: Log-loss equation with ridge penalty

Note that if you look at the cost functions for logistic regression in the scikit-learn documentation, the specific form is different than what is used here, but the overall idea is similar. Additionally, after you become comfortable with the concepts of lasso and ridge penalties, you should be aware that there is an additional regularization method called **elastic-net**, which is a combination lasso and ridge.

Why Are There Two Different Formulations of Regularization?

It may be that one or the other will provide better out-of-sample performance, so you may wish to test them both. There is another key differ-

ence in these methods: the L1 penalty also performs feature selection, in addition to regularization. It does this by setting some coefficient values to exactly zero during the regularization process, effectively removing features from the model. L2 regularization makes the coefficient values smaller but does not completely eliminate them. Not all solver options in scikit-learn support both L1 and L2 regularization, so you will need to select an appropriate solver for the regularization technique you want to use.

Note

*The mathematical details of why L1 regularization removes features but L2 doesn't are beyond the scope of this book. However, for a more thorough explanation of this topic and further reading in general, we recommend the very readable (and free) resource, *An Introduction to Statistical Learning* by Gareth James, et al. In particular, see page 222 of the corrected 7th printing, for a helpful graphic on the difference between L1 and L2 regularization.*

Intercepts and Regularization

We have not discussed intercepts very much, other than to note that we have been estimating them with our linear models, along with the coefficients that go with each feature. So, should you use an intercept? The answer is probably yes, until you've developed an advanced understanding of linear models and are certain that in a specific case you should not. However, such cases do exist, for example, in a linear regression where the features and the response variable have all been normalized to have a mean of zero.

Intercepts don't go with any particular feature. Therefore, it doesn't make much sense to regularize them, as they shouldn't contribute to overfitting. Notice that in the regularization penalty term for L1, the summation starts with $j = 1$, and similarly for L2, we have skipped σ_0 , which is the intercept term.

This is the ideal situation: not regularizing the intercept. However, some of the solvers in scikit-learn, such as **liblinear**, actually do regularize the intercept. There is an **intercept_scaling** option that you can supply to the model class to counteract this effect. We have not illustrated this here as, although it is theoretically incorrect, regularizing the intercept often does not have much effect on the model's predictive quality in practice.

Scaling and Regularization

As noted in the previous exercise, it is best practice to **scale** the data so that all the features have roughly the same range of values before using regularization. This is because the coefficients are all going to be subject to the same penalty in the cost function. If the range of values for a particular feature, such as **LIMIT_BAL** in our dataset, is much larger than other features, such as **PAY_1**, it may, in fact, be desirable to have a larger value for the coefficient of **PAY_1** and a smaller value for that of **LIMIT_BAL** in order to put their effects on the same scale in the linear combination of features and coefficients that are used for model prediction. Normalizing all the features before using regularization avoids complications such as this that arise simply from differences in scale.

In fact, scaling your data may also be necessary, depending on which solver you are using. The different variations on the gradient descent process available in scikit-learn may or may not be able to work effectively with unscaled data.

The Importance of Selecting the Right Solver

As we've come to learn, the different solvers available for logistic regression in scikit-learn have different behaviors regarding the following:

- Whether they support both L1 and L2 regularization
- How they treat the intercept during regularization
- How they deal with unscaled data

Note

There are other differences as well. A helpful table comparing these and other traits is available at [https://scikit-](https://scikit-learn.org/stable/modules/linear_model.html)

[learn.org/stable/modules/linear_model.html#logistic-regression](https://learning-oreilly-com.ezproxy.sfpl.org/library/view/data-science-projects/9781800564480/B16925_04_ePub.xhtml#logistic-regression).

You can use this table to decide which solver is appropriate for your problem.

To summarize this section, we have learned the mathematical foundations of lasso and ridge regularization. *These methods work by shrinking the coefficient values toward 0, and in the case of the lasso, setting some coefficients to exactly 0 and thus performing feature selection.* You can imagine that in our example of overfitting in Figure 4.12, if the complex, overfitted model had some coefficients shrunk toward 0, it would look more like the ideal model, which has fewer coefficients.

Here is a plot of a regularized regression model, using the same high-degree polynomial features as the overfitted model, but with a ridge penalty:

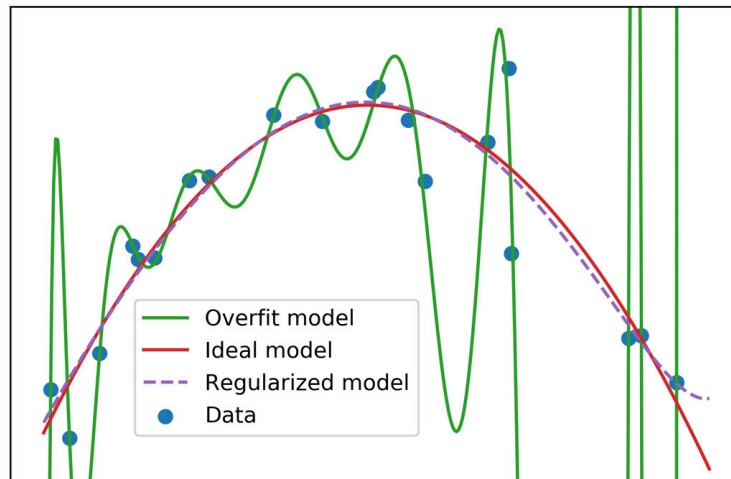


Figure 4.16: An overfit model and regularized model using the same features

The regularized model looks similar to the ideal model, demonstrating the ability of regularization to correct overfitting. Note, however, that the regularized model should not be recommended for extrapolation. Here, we can see that the regularized model starts to increase toward the right side of Figure 4.16. This increase should be viewed with suspicion, as there is nothing in the training data that makes it clear that this would be expected. This is an example of the general view that the *extrapolation of model predictions outside the range of training data is not recommended*.

However, it is clear from *Figure 4.16* that even if we didn't have knowledge of the model that was used to generate this synthetic data (as we typically don't have knowledge of the data-generating process in real-world predictive modeling work), we can still use regularization to reduce the effect of overfitting when a large number of candidate features are available.

Model and Feature Selection

L1 regularization is one way to use a model, such as logistic regression, to perform feature selection. Other methods include forward or backward **stepwise selection** from the pool of candidate features. Here is the high-level idea behind these methods: in the case of **forward selection**, features are added to the model one at a time, and the out-of-sample performance is observed along the way. At each iteration, the addition of all possible features from the candidate pool is considered, and the one resulting in the greatest increase in the out-of-sample performance is chosen. When adding additional features ceases to improve the model's performance, no more features need to be added from the candidates. In the case of **backward selection**, you first start with all the features in the model and determine which one you should remove: the one resulting in the smallest decrease in the out-of-sample performance. You can continue removing features in this way until the performance begins to decrease appreciably.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/aUBMb>.

Cross-Validation: Choosing the Regularization Parameter

By now, you may suspect that we could use regularization in order to decrease the overfitting we observed when we tried to model the synthetic

data in *Exercise 4.02, Generating and Modeling Synthetic Classification Data*. The question is, how do we choose the regularization parameter, C ? C is an example of a model **hyperparameter**. Hyperparameters are different from the parameters that are estimated when a model is trained, such as the coefficients and the intercept of a logistic regression. Rather than being estimated by an automated procedure like the parameters are, hyperparameters are input directly by the user as keyword arguments, typically when instantiating the model class. So, how do we know what values to choose?

Hyperparameters are more difficult to estimate than parameters. This is because it is up to the data scientist to determine what the best value is, as opposed to letting an optimization algorithm find it. However, it is possible to programmatically choose hyperparameter values, which could be viewed as an optimization procedure in its own right. Practically speaking, in the case of the regularization parameter C , this is most commonly done by fitting the model on one set of data with a particular value of C , determining model training performance, and then assessing the out-of-sample performance on another set of data.

We are already familiar with the concept of using model training and test sets. However, there is a key difference here; for instance, what would happen if we were to use the test set multiple times in order to see the effect of different values of C ?

It may occur to you that after the first time you use the unseen test set to assess the out-of-sample performance for a particular value of C , it is no longer an "unseen" test set. While only the training data was used for estimating the model parameters (that is, the coefficients and the intercept), now the test data is being used to estimate the hyperparameter C . Effectively, the test data has now become additional training data in the sense that it is being used to find a good value for the hyperparameter.

For this reason, it is common to divide the data into three parts: a training set, a test set, and a **validation set**. The validation set serves multiple purposes:

Estimating Hyperparameters

The validation set can be repeatedly used to assess the out-of-sample performance with different hyperparameter values to select hyperparameters.

A Comparison of Different Models

In addition to finding hyperparameter values for a model, the validation set can be used to estimate the out-of-sample performance of different models; for example, if we wanted to compare logistic regression to random forest.

Note

Data Management Best Practices

*As a data scientist, it's up to you to figure out how to divide up your data for different predictive modeling tasks. In the ideal case, you should reserve a portion of your data for the very end of the process, after you've already selected model hyperparameters and also selected the best model. This **unseen test set** is reserved for the last step, when it can be used to assess the endpoint of your model-building efforts, to see how the final model generalizes to new unseen data. When reserving the test set, it is good practice to make sure that the features and responses have similar characteristics to the rest of the data. In other words, the class fraction should be the same, and the distribution of features should be similar. This way, the test data should be representative of the data you built the model with.*

While model validation is a good practice, it raises the question of whether the particular split we choose for the training, validation, and test data has any effect on the outcomes that we are tracking. For example, perhaps the relationship between the features and the response variable is slightly different in the unseen test set that we have reserved, or in the validation set, versus the training set. It is likely impossible to eliminate all such variability, but we can use the method of **cross-validation** to avoid placing too much faith in one particular split of the data.

Scikit-learn provides convenient functions to facilitate cross-validation analyses. These functions play a similar role to `train_test_split`, which we have already been using, although the default behavior is somewhat different. Let's get familiar with them now. First, import these two classes:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import KFold
```

Similar to `train_test_split`, we need to specify what proportion of the dataset we would like to use for training versus testing. However, with cross-validation (specifically the **k-fold cross-validation** that was implemented in the classes we just imported), rather than specifying a proportion directly, we simply indicate how many folds we would like – that is, the "**k folds**." The idea here is that the data will be divided into **k** equal proportions. For example, if we specify 4 folds, then each fold will have 25% of the data. These folds will be the test data in four separate instances of model training, while the remaining 75% from each fold will be used to train the model. In this procedure, each data point gets used as training data a total of $k - 1$ times, and as test data only once.

When instantiating the class, we indicate the number of folds, whether or not to shuffle the data before splitting, and a random seed if we want repeatable results across different runs:

```
n_folds = 4
k_folds = KFold(n_splits=n_folds, shuffle=False)
```

Here, we've instantiated an object with four folds and no shuffling. The way in which we use the object that is returned, which we've called **k_folds**, is by passing the features and response data that we wish to use for cross-validation, to the `.split` method of this object. This outputs an **iterator**, which means that we can loop through the output to get the different splits of training and test data. If we took the training data from our synthetic classification problem, `X_syn_train` and `y_syn_train`, we could loop through the splits like this:

```

for train_index, test_index in
    k_folds_iterator.split(X_syn_train,
                           y_syn,
                           n_train):

```

The iterator will return the row indices of **X_syn_train** and **y_syn_train**, which we can use to index the data. Inside this **for** loop, we can write code to use these indices to select data for repeatedly training and testing a model object with different subsets of the data. In this way, we can get a robust indication of the out-of-sample performance when using one particular hyperparameter value, and then repeat the whole process using another hyperparameter value. Consequently, the cross-validation loop may sit **nested** inside an outer loop over different hyperparameter values. We'll illustrate this in the following exercise.

First though, what do these splits look like? If we were to simply plot the indices from **train_index** and **test_index** as different colors, we would get something that looks like this:

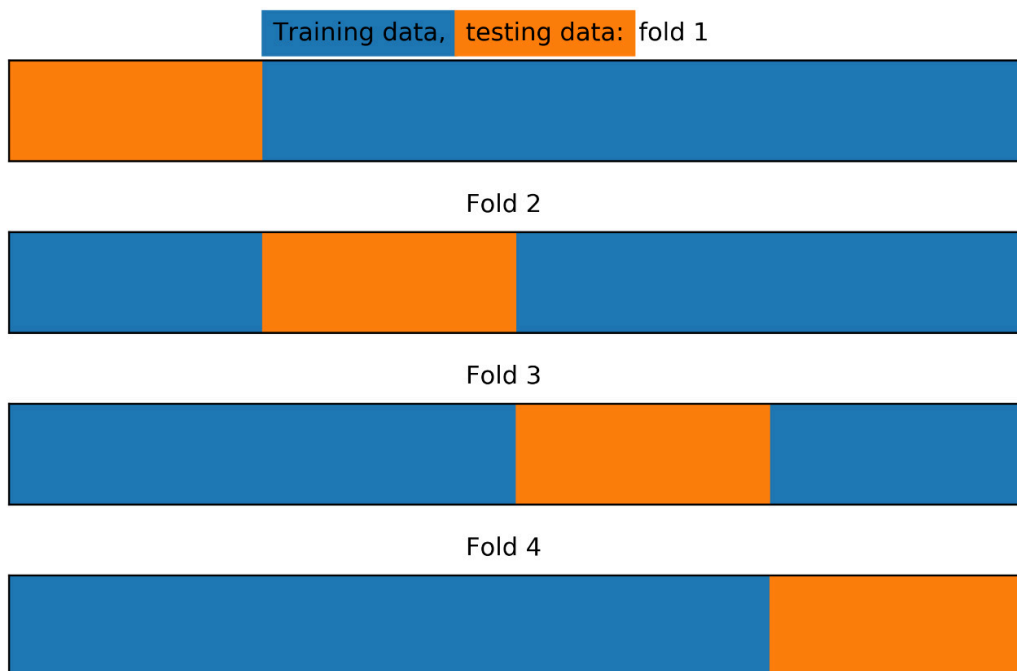


Figure 4.17: Training/test splits for k-folds with four folds and no shuffling

Here, we see that with the options we've indicated for the **KFold** class, the procedure has simply taken the first 25% of the data, according to the order of rows, as the first test fold, then the next 25% of data for the second

fold, and so on. But what if we wanted stratified folds? In other words, what if we wanted to ensure that the class fractions of the response variable were equal in every fold? While `train_test_split` allows this option as a keyword argument, there is a separate `StratifiedKFold` class that implements this for cross-validation. We can illustrate how the stratified splits will appear as follows:

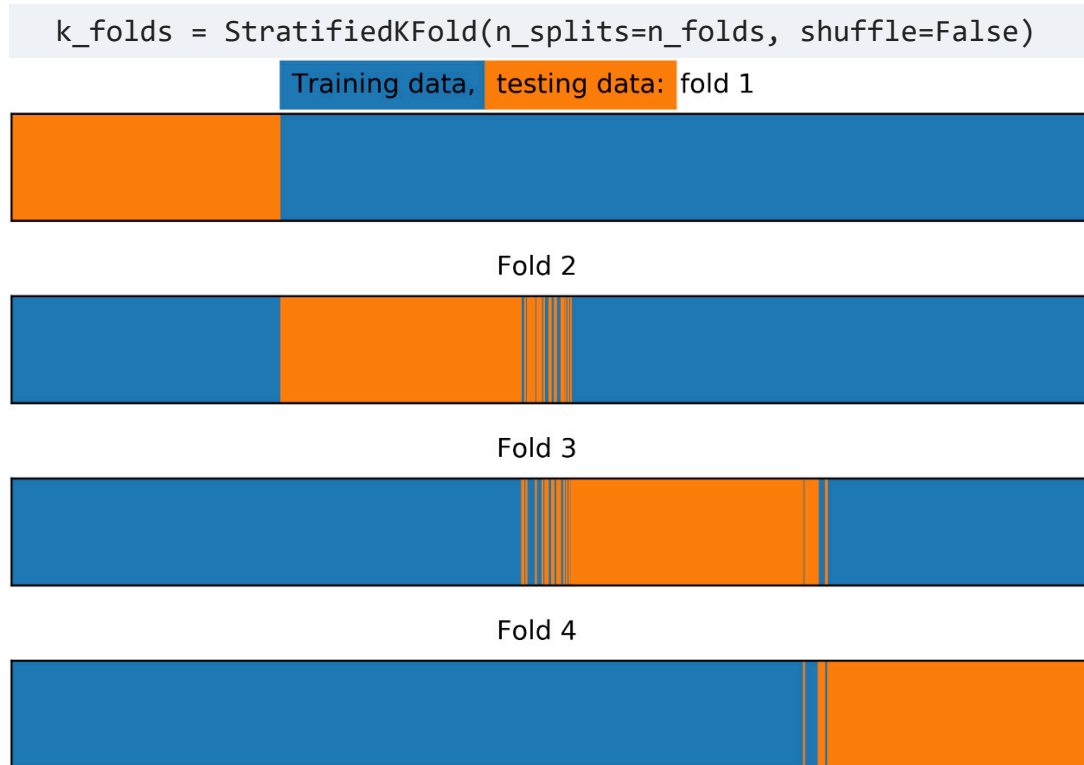


Figure 4.18: Training/test splits for stratified k-folds

In *Figure 4.18*, we can see that there has been some amount of "shuffling" between the different folds. The procedure has moved samples between folds as necessary to ensure that the class fractions in each fold are equal.

Now, what if we want to shuffle the data to choose samples from throughout the range of indices for each test fold? First, why might we want to do this? Well, with the synthetic data that we've created for our problem, we can be certain that the data is in no particular order. However, in many real-world situations, the data we receive may be sorted in some way.

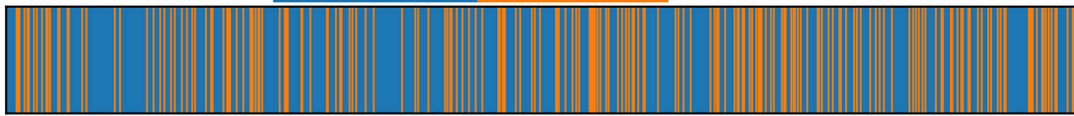
For instance, perhaps the rows of the data have been ordered by the date an account was created, or by some other logic. Therefore, it can be a good idea to shuffle the data before splitting. This way, any traits that

might have been used for sorting can be expected to be consistent throughout the folds. Otherwise, the data in different folds may have different characteristics, possibly leading to different relationships between features and response.

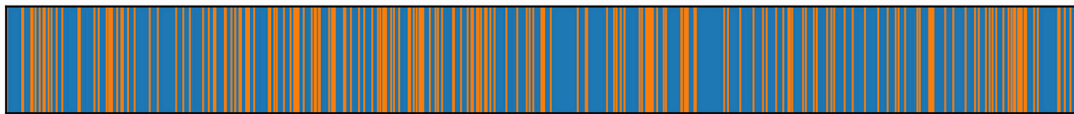
This can lead to a situation where model performance is uneven between the folds. In order to "mix up" the folds throughout all the row indices of a dataset, all we need to do is set the **shuffle** parameter to **True**:

```
k_folds = StratifiedKFold(n_splits=n_folds, shuffle=True,  
                           random_state=1)
```

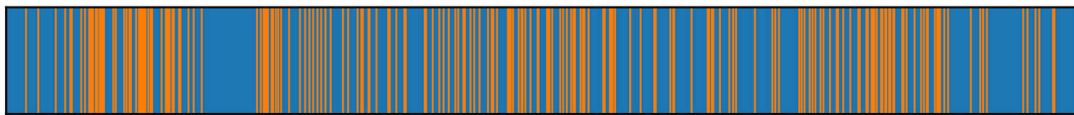
Training data, testing data: fold 1



Fold 2



Fold 3



Fold 4

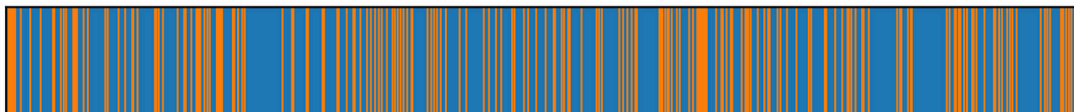


Figure 4.19: Training/test splits for stratified k-folds with shuffling

With shuffling, the test folds are spread out randomly, and fairly evenly, across the indices of the input data.

K-fold cross-validation is a widely used method in data science. However, the choice of how many folds to use depends on the particular dataset at hand. Using a smaller number of folds means that the amount of training data in each fold will be relatively small. Therefore, this increases the chances that the model will underfit, as models generally work better when trained on more data. It's a good idea to try a few different numbers of folds and see how the mean and the variability of the k-fold test

score changes. Common numbers of folds can range anywhere from 4 or 5 to 10.

In the event of a very small dataset, it may be necessary to use as much data as possible for training in the cross-validation folds. In this scenario, you can use a method called **leave-one-out cross-validation (LOOCV)**. In LOOCV, the test set for each fold consists of a single sample. In other words, there will be as many folds as there are samples in the training data. For each iteration, the model is trained on all but one sample, and a prediction is made for that sample. The accuracy, or other performance metric, can then be constructed using these predictions.

Other concerns that relate to the creation of a test set, such as choosing an out-of-time test set for problems where observations from the past must be used to predict future events, also apply to cross-validation.

In *Exercise 4.02, Generating and Modeling Synthetic Classification Data*, we saw that fitting a logistic regression on our training data led to overfitting. Indeed, the test score ($ROC\ AUC = 0.81$) was substantially lower than the training score ($ROC\ AUC = 0.94$). We had essentially used very little or no regularization by setting the regularization parameter C to a relatively large value (1,000). Now we will see what happens when we vary C through a wide range of values.

Note

The code for generating the plots presented in this section can be found here: <https://packt.link/37Zks>.

Exercise 4.03: Reducing Overfitting on the Synthetic Data Classification Problem

This exercise is a continuation of *Exercise 4.02, Generating and Modeling Synthetic Classification Data*. Here, we will use a cross-validation procedure in order to find a good value for the hyperparameter C . We will do this by using only the training data, reserving the test data for after model building is complete. Be prepared – this is a long exercise – but it

will illustrate a general procedure that you will be able to use with many different kinds of machine learning models, so it is worth the time spent here. Perform the following steps to complete the exercise:

Note

Before you begin this exercise, you need to execute some prerequisite steps that can be found in the following notebook along with the code for this exercise: <https://packt.link/JqbsW>.

1. Vary the value of the regularization parameter, C , to range from $C = 1000$ to $C = 0.001$. You can use the following snippets to do this.

First, define exponents, which will be powers of 10, as follows:

```
C_val_exponents = np.linspace(3, -3, 13)
C_val_exponents
```

Here is the output of the preceding code:

```
array([ 3. , 2.5, 2. , 1.5, 1. , 0.5, 0. , -0.5, -1. ,
       -1.5, -2. , -2.5, -3. ])
```

Now, vary C by the powers of 10, as follows:

```
C_vals = np.float(10)**C_val_exponents
C_vals
```

Here is the output of the preceding code:

```
array([1.00000000e+03, 3.16227766e+02, 1.00000000e+02,
       3.16227766e+01,
       1.00000000e+01, 3.16227766e+00, 1.00000000e+00,
       3.16227766e-01,
       1.00000000e-01, 3.16227766e-02, 1.00000000e-02,
       3.16227766e-03,
       1.00000000e-03])
```

It's generally a good idea to vary the regularization parameter by powers of 10, or by using a similar strategy, as training models can take a substantial amount of time, especially when using k-fold cross-validation. This gives you a good idea of how a wide range of C values impacts the bias-variance trade-off, without needing to train a very large number of models. In addition to the integer powers of 10, we also include points on the $\log_{10}$ scale that are about halfway between. If it

seems like there is some interesting behavior in between these relatively widely spaced values, you can add more granular values for C in a smaller part of the range of possible values.

2. Import the `roc_curve` class:

```
from sklearn.metrics import roc_curve
```

We'll continue to use the ROC AUC score for assessing, training, and testing performance. Now that we have several values of C to try and several folds (in this case four) for the cross-validation, we will want to store the training and test scores for each fold and for each value of C .

3. Define a function that takes the `k_folds` cross-validation splitter, the array of C values (`C_vals`), the model object (`model`), and the features and response variable (`X` and `Y`, respectively) as inputs, to explore different amounts of regularization with k-fold cross-validation. Use the following code:

```
def cross_val_C_search(k_folds, C_vals, model, X, Y):
```

Note

The function we started in this step will return the ROC AUCs and ROC curve data. The return block will be written during a later step in the exercise. For now, you can simply write the preceding code as is, because we will be defining `k_folds`, `C_vals`, `model`, `X`, and `Y` as we progress in the exercise.

4. Within this function block, create a NumPy array to hold model performance data, with dimensions `n_folds` by `len(C_vals)`:

```
n_folds = k_folds.n_splits
cv_train_roc_auc = np.empty((n_folds, len(C_vals)))
cv_test_roc_auc = np.empty((n_folds, len(C_vals)))
```

Next, we'll store the arrays of true and false positive rates and thresholds that go along with each of the test ROC AUC scores in a **list of lists**.

Note

*This is a convenient way to store all this model performance information, as a list in Python can contain any kind of data, including another list. Here, each item of the inner lists in the **list of lists** will be a tuple holding the arrays of TPR, FPR, and the thresholds for each of the folds, for each of the C values. Tuples are an ordered collection data type in*

Python, similar to lists, but unlike lists they are immutable: the items in a tuple can't be changed after the tuple is created. When a function returns multiple values, like the `roc_curve` function of `scikit-learn`, these values can be output to a single variable, which will be a tuple of those values. This way of storing results should be more obvious when we access these arrays later in order to examine them.

5. Create a list of empty lists using `[[[]]]` and `*len(C_vals)` as follows:

```
cv_test_roc = [[]]*len(C_vals)
```

Using `*len(C_vals)` indicates that there should be a list of tuples of metrics (TPR, FPR, thresholds) for each value of `C`.

We have learned how to loop through the different folds for cross-validation in the preceding section. What we need to do now is write an outer loop in which we will nest the cross-validation loop.

6. Create an outer loop for training and testing each of the `k`-folds for each value of `C`:

```
for c_val_counter in range(len(C_vals)):
    #Set the C value for the model object
    model.C = C_vals[c_val_counter]
    #Count folds for each value of C
    fold_counter = 0
```

We can reuse the same model object that we have already, and simply set a new value of `C` within each run of the loop. Inside the loop of `C` values, we run the cross-validation loop. We begin by yielding the training and test data row indices for each split.

7. Obtain the training and test indices for each fold:

```
for train_index, test_index in k_folds.split(X, Y):
```

8. Index the features and response variable to obtain the training and test data for this fold using the following code:

```
X_cv_train, X_cv_test = X[train_index], X[test_index]
y_cv_train, y_cv_test = Y[train_index], Y[test_index]
```

The training data for the current fold is then used to train the model.

9. Fit the model on the training data, as follows:

```
model.fit(X_cv_train, y_cv_train)
```

This will effectively "reset" the model from whatever the previous coefficients and intercept were to reflect the training on this new data.

The training and test ROC AUC scores are then obtained, as well as the arrays of TPRs, FPRs, and thresholds that go along with the test data.

10. Obtain the training ROC AUC score:

```
y_cv_train_predict_proba =
model.predict_proba(X_cv_train)
cv_train_roc_auc[fold_counter, c_val_counter] = \
roc_auc_score(y_cv_train,
y_cv_train_predict_proba[:,1])
```

11. Obtain the test ROC AUC score:

```
y_cv_test_predict_proba =
model.predict_proba(X_cv_test)
cv_test_roc_auc[fold_counter, c_val_counter] = \
roc_auc_score(y_cv_test, y_cv_test_predict_proba[:,1])
```

12. Obtain the test ROC curves for each fold using the following code:

```
this_fold_roc = roc_curve(y_cv_test,
y_cv_test_predict_proba[:,1])
cv_test_roc[c_val_counter].append(this_fold_roc)
```

We will use a fold counter to keep track of the folds that are incremented, and once outside the cross-validation loop, we print a status update to standard output. Whenever performing long computational procedures, it's a good idea to periodically print the status of the job so that you can monitor its progress and confirm that things are still working correctly. This cross-validation procedure will likely take only a few seconds on your laptop, but for longer jobs this can be especially reassuring.

13. Increment the fold counter using the following code:

```
fold_counter += 1
```

14. Write the following code to indicate the progress of execution for each value of C:

```
print('Done with C = {}'.format(lr_syn.C))
```

15. Write the code to return the ROC AUCs and ROC curve data and finish the function:

```
return cv_train_roc_auc, cv_test_roc_auc, cv_test_roc
```

Note that we will continue to use the split into four folds that we illustrated previously, but you are encouraged to try this procedure with different numbers of folds to compare the effect.

We have covered a lot of material in the preceding steps. You may want to take a few moments to review this with your classmates in order to make sure that you understand each part. Running the function is comparatively simple. That is the beauty of a well-designed function – all the complicated parts get abstracted away, allowing you to concentrate on usage.

16. Run the function we've designed to examine cross-validation performance, with the C values that we previously defined, and by using the model and data we were working with in the previous exercise. Use the following code:

```
cv_train_roc_auc, cv_test_roc_auc, cv_test_roc = \
    cross_val_C_search(k_folds, C_vals, lr_syn,
                      X_syn_train, y_syn_train)
```

When you run this code, you should see the following output populate below the code cell as the cross-validation is completed for each value of C :

```
Done with C = 1000.0
Done with C = 316.22776601683796
Done with C = 100.0
Done with C = 31.622776601683793
Done with C = 10.0
Done with C = 3.1622776601683795
Done with C = 1.0
Done with C = 0.31622776601683794
Done with C = 0.1
Done with C = 0.03162277660168379
Done with C = 0.01
Done with C = 0.0031622776601683794
Done with C = 0.001
```

So, what do the results of the cross-validation look like? There are a few ways to examine this. It is useful to look at the performance of each fold individually, so that you can see how variable the results are. This tells you how different subsets of your data perform as test sets, leading to a general idea of the range of performance you can expect from the unseen test set. What we're interested in here is whether or not we are able to use regularization to alleviate the overfitting that

we saw. We know that using $C = 1,000$ led to overfitting – we know this from comparing the training and test scores. But what about the other C values that we've tried? A good way to visualize this will be to plot the training and test scores on the y -axis and the values of C on the x -axis.

17. Loop over each of the folds to view their results individually by using the following code:

```
for this_fold in range(k_folds.n_splits):
    plt.plot(C_val_exponents,
             cv_train_roc_auc[this_fold], '-o',\
             color=cmap(this_fold),\
             label='Training fold'
             '{}'.format(this_fold+1))
    plt.plot(C_val_exponents,
             cv_test_roc_auc[this_fold], '-x',\
             color=cmap(this_fold),\
             label='Testing fold'
             '{}'.format(this_fold+1))
plt.ylabel('ROC AUC')
plt.xlabel('log$_{10}$$(C)$')
plt.legend(loc = [1.1, 0.2])
plt.title('Cross validation scores for each fold')
```

You will obtain the following output:

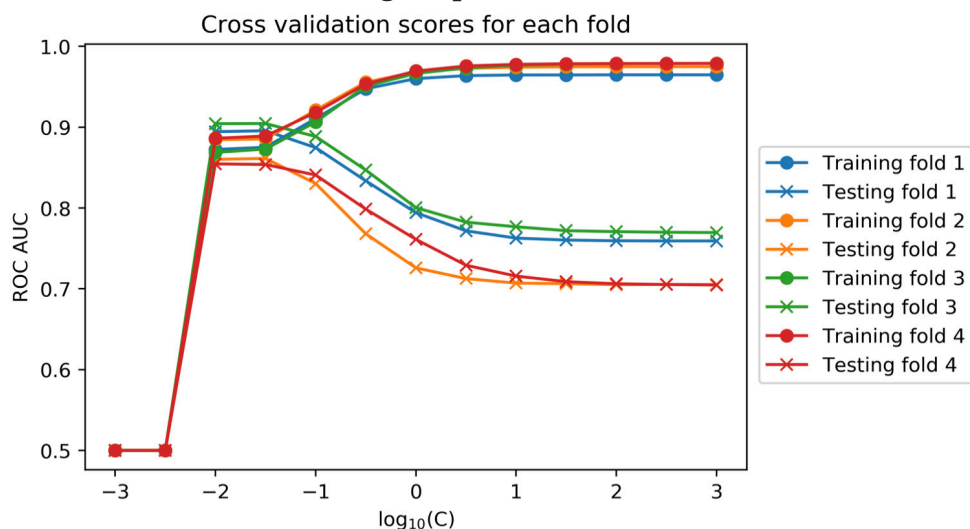


Figure 4.20: The training and test scores for each fold and C -value
We can see that for each fold of the cross-validation, as C decreases, the training performance also decreases. However, at the same time,

the test performance increases. For some folds and values of C , the test ROC AUC score actually exceeds that of the training data, while for others, these two metrics simply come closer together. In all cases, we can say that the C values of $10^{-1.5}$ and 10^{-2} appear to have a similar test performance, which is substantially higher than the test performance of $C = 10^3$. So, it appears that regularization has successfully addressed our overfitting problem.

But what about the lower values of C ? For values that are lower than 10^{-2} , the ROC AUC metric suddenly drops to 0.5. As you know, this value means that the classification model is essentially useless, performing no better than a coin flip. You are encouraged to check on this later when exploring how regularization affects the coefficient values; however, this is what happens when so much L1 regularization is applied that all model coefficients shrink to 0. Obviously, such models are not useful to us, as they encode no information about the relationship between the features and response variable.

Looking at the training and test performance of each k-fold split is helpful for gaining insights into the variability of model performance that may be expected when the model is scored on new, unseen data. But in order to summarize the results of the k-folds procedure, a common approach is to average the performance metric over the folds, for each value of the hyperparameter being considered. We'll perform this in the next step.

18. Plot the mean of training and test ROC AUC scores for each C value using the following code:

```
plt.plot(C_val_exponents, np.mean(cv_train_roc_auc,
axis=0), \
        '-o', label='Average training score')
plt.plot(C_val_exponents, np.mean(cv_test_roc_auc,
axis=0), \
        '-x', label='Average testing score')
plt.ylabel('ROC AUC')
plt.xlabel('log10$(C)$')
plt.legend()
plt.title('Cross validation scores averaged over all
folds')
```

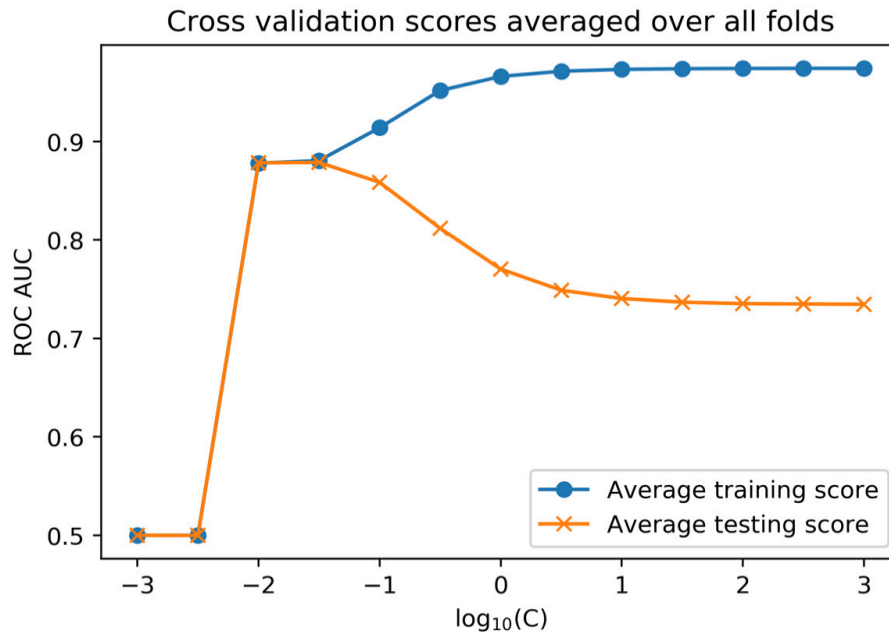


Figure 4.21: The average training and test scores across cross-validation folds

From this plot, it's clear that $C = 10^{-1.5}$ and 10^{-2} are the best values of C . There is little or no overfitting here, as the average training and test scores are nearly the same. You could search a finer grid of C values (that is $C = 10^{-1.1}$, $10^{-1.2}$, and so on) in order to more precisely locate a C value. However, from our graph, we can see that either $C = 10^{-1.5}$ or $C = 10^{-2}$ will likely be good solutions. We will move forward with $C = 10^{-1.5}$. Examining the summary metric of ROC AUC is a good way to get a quick idea of how models will perform. However, for any real-world business application, you will often need to choose a specific threshold, which goes along with specific true and false positive rates. These will be needed to use the classifier to make the required "yes" or "no" decision, which, in our case study, is a prediction of whether an account will default. For this reason, it is useful to look at the ROC curves across the different folds of the cross-validation. To facilitate this, the preceding function has been designed to return the true and false positive rates, and thresholds, for each test fold and value of C , in the `cv_test_roc` list of lists. First, we need to find the index of the outer list that corresponds to the C value that we've chosen, $10^{-1.5}$.

To accomplish this, we could simply look at our list of C values and count by hand, but it's safer to do this programmatically by finding the index of the non-zero element of a Boolean array, as is shown in the next step.

19. Use a Boolean array to find the index where $C = 10^{-1.5}$ and convert it to an integer data type with the following code:

```
best_C_val_bool = C_val_exponents == -1.5
best_C_val_bool.astype(int)
```

Here is the output of the preceding code:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0])
```

20. Convert the integer version of the Boolean array into a single integer index using the **nonzero** function with this code:

```
best_C_val_ix = np.nonzero(best_C_val_bool.astype(int))
best_C_val_ix[0][0]
```

Here is the output of the preceding code:

```
9
```

We have now successfully located the C value that we wish to use.

21. Access the true and false positive rates in order to plot the ROC curves for each fold:

```
for this_fold in range(k_folds_n_splits):
    fpr = cv_test_roc[best_C_val_ix[0][0]][this_fold]
    [0]
    tpr = cv_test_roc[best_C_val_ix[0][0]][this_fold]
    [1]
    plt.plot(fpr, tpr, label='Fold
    {}'.format(this_fold+1))
    plt.xlabel('False positive rate')
    plt.ylabel('True positive rate')
    plt.title('ROC curves for each fold at C =
    $10^{-1.5}$')
    plt.legend()
```

You will obtain the following output:

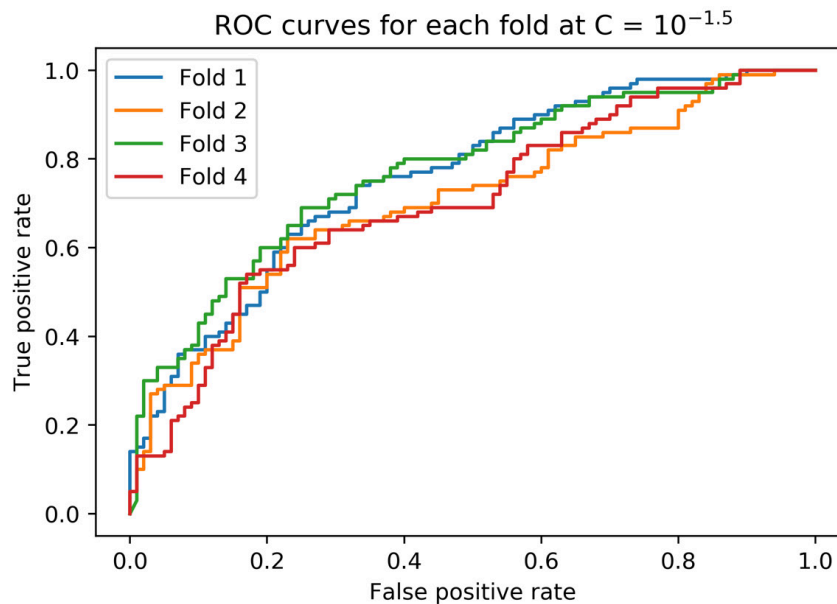


Figure 4.22: ROC curves for each fold

It appears that there is a fair amount of variability in the ROC curves. For example, if, for some reason, we want to limit the false positive rate to 40%, then from the plot it appears that we may be able to achieve a true positive rate of anywhere from approximately 60% to 80%. You can find the exact values by examining the arrays that we have plotted. This gives you an idea of how much variability in performance can be expected when deploying the model on new data. Generally, the more training data that is available, then the less variability there will be between the folds of cross-validation, so this could also be a sign that it would be a good idea to collect additional data, especially if the variability between training folds seems unacceptably high. You also may wish to try different numbers of folds with this procedure so as to see the effect on the variability of results between folds.

While normally we would try other models on our synthetic data problem, such as a random forest or support vector machine, if we imagine that in cross-validation, logistic regression proved to be the best model, we would decide to make this our final choice. When the final model is selected, all the training data can be used to fit the model, using the hyperparameters chosen with cross-validation. It's best to use as much data as possible in model fitting, as models typically work better when trained on more data.

22. Train the logistic regression on all the training data from our synthetic problem and compare the training and test scores, using the held-out

test set as shown in the following steps.

Note

This is the final step in the model selection process. You should only use the unseen test set after your choice of model and hyperparameters are considered finished, otherwise it will not be "unseen."

23. Set the C value and train the model on all the training data with this code:

```
lr_syn.C = 10**(-1.5)
lr_syn.fit(X_syn_train, y_syn_train)
```

Here is the output of the preceding code:

```
LogisticRegression(C=0.03162277660168379, penalty='l1',
\
                      random_state=1, solver='liblinear'))
```

24. Obtain predicted probabilities and the ROC AUC score for the training data with this code:

```
y_syn_train_predict_proba =
lr_syn.predict_proba(X_syn_train)
roc_auc_score(y_syn_train,
y_syn_train_predict_proba[:,1])
```

Here is the output of the preceding code:

```
0.8802812499999999
```

25. Obtain predicted probabilities and the ROC AUC score for the test data with this code:

```
y_syn_test_predict_proba =
lr_syn.predict_proba(X_syn_test)
roc_auc_score(y_syn_test,
y_syn_test_predict_proba[:,1])
```

Here is the output of the preceding code:

```
0.8847884788478848
```

Here, we can see that by using regularization, the model training and test scores are similar, indicating that the overfitting problem has been greatly reduced. The training score is lower since we have introduced bias into the model at the expense of variance. However, this is OK, since the test score, which is the most important part, is higher. The out-of-sample test score is what matters for predictive capability. You are encouraged to check that these training and test scores are

similar to those from the cross-validation procedure by printing the values from the arrays that we plotted previously; you should find that they are.

Note

In a real-world project, before delivering this model to your client for production use, you may wish to train the model on all the data that you were given, including the unseen test set. This follows the idea that the more data a model has seen, the better it is likely to perform in practice. However, some practitioners prefer to only use models that have been tested, meaning you would deliver the model trained only on the training data, not including the test set.

We know that L1 regularization works by decreasing the magnitude (that is, absolute value) of coefficients of the logistic regression. It can also set some coefficients to zero, therefore performing feature selection. In the next step, we will determine how many coefficients were set to zero.

26. Access the coefficients of the trained model and determine how many do not equal zero (`!= 0`) with this code:

```
sum((lr_syn.coef_ != 0)[0])
```

The output should be as follows:

```
2
```

This code takes the sum of a Boolean array indicating the locations of non-zero coefficients, so it shows how many coefficients in the model did not get set to zero by L1 regularization. Only 2 of the 200 features were selected!

27. Examine the value of the intercept using this code:

```
lr_syn.intercept_
```

The output should be as follows:

```
array([0.])
```

This shows that the intercept was regularized to 0.

In this exercise, we accomplished several goals. We used the k-fold cross-validation procedure to tune the regularization hyperparameter. We saw the power of regularization for reducing overfitting, and in the case of L1 regularization in logistic regression, selecting features.

Many machine learning algorithms offer some type of feature selection capability. Many also require the tuning of hyperparameters. The function here that loops over hyperparameters, and performs cross-validation, is a powerful concept that generalizes to other models. Scikit-learn offers functionality to make this process easier; in particular, the `sklearn.model_selection.GridSearchCV` procedure, which applies cross-validation to a grid search over hyperparameters. A **grid search** can be helpful when there are multiple hyperparameters to tune, by looking at all combinations of the ranges of different hyperparameters that you specify. A **randomized grid search** can speed up this process by randomly choosing a smaller number of combinations when an exhaustive grid search would take too long. Once you are comfortable with the concepts illustrated here, you are encouraged to streamline your workflow with convenient functions like these.

Options for Logistic Regression in Scikit-Learn

We have used and discussed most of the options that you may supply to scikit-learn when instantiating or tuning the hyperparameters of a **LogisticRegression** model class. Here, we list them all and provide some general advice on their usage:

Parameter	Possible values	Notes and advice for choosing
penalty	string, 'l1', 'l2', 'elasticnet', or 'none'	L1 (lasso) or L2 (ridge) regularization of coefficients. L1 performs feature selection, while L2 does not. Elastic-net is a blend of L1 and L2. The best overall model performance should be assessed by trying all options.
dual	bool, True, or False	This has to do with the optimization algorithm used to find coefficients. The documentation says <i>"only implemented for l2 penalty with liblinear solver. Prefer dual=False when n_samples > n_features."</i>
tol	float (decimal number)	Determines the size of the change in values for the optimization algorithm to stop. This is one way to control how long the optimization runs for, and how close to the ideal value the solution is.
C	float	The regularization parameter for L1 or L2 penalties. Smaller values mean more regularization. This needs to be determined using a validation set, or cross-validation.
fit_intercept	Bool	Whether or not an intercept term should be estimated. Unless you are sure you don't need an intercept, it's probably best to have one.
intercept_scaling	float	Can be used to avoid regularizing the intercept, an undesirable practice, when using the liblinear solver.
class_weight	Dictionary-specify weight for each class, 'balanced' string, or None	Whether or not to weight different classes during the model training process. Otherwise all samples will be considered equally important when fitting the model. Can be useful for imbalanced datasets: try using 'balanced' in this case.
random_state	int	Seed for a random number generator used by certain solver algorithms.
solver	string ('newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga')	Select the type of optimization algorithm used to estimate the model parameters. See earlier discussion in this chapter or the documentation for the relative strengths and weaknesses of different solvers.
max_iter	int	The maximum number of iterations for the solution algorithm, which controls how close to the ideal parameters the solution is. If you get a warning that the solution algorithm did not converge, you can try increasing this.
multi_class	string ('ovr', 'multinomial', 'auto')	Various strategies for multiclass classification, beyond the scope of this book.
verbose	int	Controls the nature of the output to the terminal, during the optimization procedure.
warm_start	bool	If re-using the same model object for multiple training iterations, whether or not to use the previous solution as the starting point for the next optimization procedure.
n_jobs	int or None	Number of processors to use for parallel processing, in the case of 'ovr' multiclass classification.
l1_ratio	float	A parameter controlling the relative contributions of L1 and L2 regularization when using the elastic-net penalty.

Figure 4.23: A complete list of options for the logistic regression model in scikit-learn

If you are in doubt regarding which option to use for logistic regression, we recommend you consult the scikit-learn documentation for further guidance (https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression).

Some options, such as the regularization parameter *C*, or the choice of a penalty for regularization, will need to be explored through the cross-validation process. Here, as with many choices to be made in data science, there is no universal approach that will apply to all datasets. The best way to see which options to use with a given dataset is to try several of them and see which gives the best out-of-sample performance. Cross-validation offers you a robust way to do this.

Scaling Data, Pipelines, and Interaction Features in Scikit-Learn

Scaling Data

Compared to the synthetic data we were just working with, the case study data is relatively large. If we want to use L1 regularization, then according to the official documentation (https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression), we ought to use the **saga** solver. However, this solver is not robust to unscaled datasets. Hence, we need to be sure to scale the data. This is also a good idea whenever doing regularization, so all the features are on the same scale and are equally penalized by the regularization process. A simple way to make sure that all the features have the same scale is to put them all through the transformation of subtracting the minimum and dividing by the range from minimum to maximum. This transforms each feature so that it will have a minimum of 0 and a maximum of 1. To instantiate the **MinMaxScaler** scaler that does this, we can use the following code:

```
from sklearn.preprocessing import MinMaxScaler
min_max_sc = MinMaxScaler()
```

Pipelines

Previously, we used a logistic regression model in the cross-validation loop. However, now that we're scaling data, what new considerations are there? The scaling is effectively "learned" from the minimum and maximum values of the training data. After this, a logistic regression model would be trained on data scaled by the extremes of the model training data. However, we won't know the minimum and maximum values of the new, unseen data. So, following the philosophy of making cross-validation an effective indicator of model performance on unseen data, we need to use the minimum and maximum values of the training data in each cross-validation fold in order to scale the test data in that fold, before making predictions on the test data. Scikit-learn has the functionality to facilitate the combination of several training and test steps for situations such as this: the **Pipeline**. Our pipeline will consist of two steps: the scaler and the logistic regression model. These can both be fit on the training data and then used to make predictions on the test data. The process of fitting

a pipeline is executed as a single step in the code, so all the parts of the pipeline are fit at once in this sense. Here is how a **Pipeline** is instantiated:

```
from sklearn.pipeline import Pipeline
scale_lr_pipeline = Pipeline(steps=[('scaler',
min_max_sc), \
                                   ('model', lr)])
```

Interaction Features

Considering the case study data, do you think a logistic regression model with all possible features would be overfit or underfit? You can think about this from the perspective of rules of thumb, such as the "rule of 10," and the number of features (17) versus samples (26,664) that we have. Alternatively, you can consider all the work we've done so far with this data. For instance, we've had a chance to visualize all the features and ensure they make sense. Since there are relatively few features, and we have relatively high confidence that they are high quality because of our data exploration work, we are in a different situation than with the synthetic data exercises in this chapter, where we had a large number of features about which we knew relatively little. So, it may be that overfitting will be less of an issue with our case study at this point, and the benefits of regularization may not be significant.

In fact, it may be that we will underfit the model using only the 17 features that came with the data. One strategy to deal with this is to engineer new features. Some simple feature engineering techniques we've discussed include interaction and polynomial features. Polynomials may not make sense given the way in which some of the data has been encoded; for example, $-1^2 = 1$, which may not be sensible for **PAY_1**. However, we may wish to try creating interaction features to capture the relationships between features. **PolynomialFeatures** can be used to create interaction features only, without polynomial features. The example code is as follows:

```
make_interactions = PolynomialFeatures(degree=2, \
```

```
interaction_only=True, \n\ninclude_bias=False)
```

Here, **degree** represents the degree of the polynomial features, **interaction_only** takes a Boolean value (setting it to **True** indicates that only interaction features will be created), and so does **include_bias**, which adds an intercept to the model (the default value is **False**, which is correct here as the logistic regression model will add an intercept).

Activity 4.01: Cross-Validation and Feature Engineering with the Case Study Data

In this activity, we'll apply the knowledge of cross-validation and regularization that we've learned in this chapter to the case study data. We'll perform basic feature engineering. In order to estimate parameters for the regularized logistic regression model for the case study data, which is larger in size than the synthetic data that we've worked with, we'll use the **saga** solver. In order to use this solver, and for the purpose of regularization, we'll need to **scale** our data as part of the modeling process, leading us to the use of **Pipeline** class in scikit-learn. Once you have completed the activity, you should obtain an improved cross-validation test performance with the use of interaction features, as shown in the following diagram:

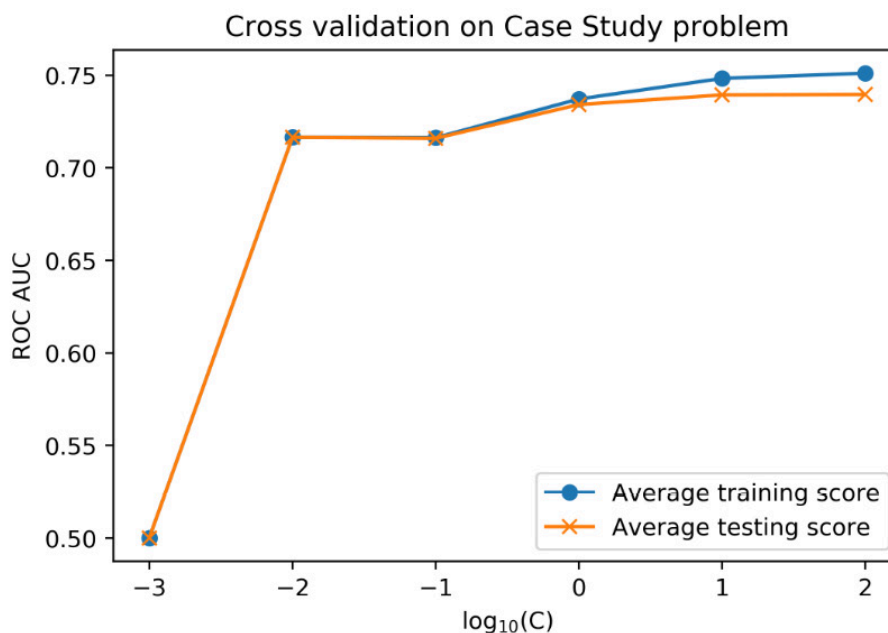


Figure 4.24: Improved model test performance

Perform the following steps to complete the activity:

1. Select the features from the DataFrame of the case study data.
You can use the list of feature names that we've already created in this chapter, but be sure not to include the response variable, which would be a very good (but entirely inappropriate) feature!
2. Make a training/test split using a random seed of 24.
We'll use this going forward and reserve this test data as the unseen test set. By specifying the random seed, we can easily create separate notebooks with other modeling approaches using the same training data.
3. Instantiate **MinMaxScaler** to scale the data.
4. Instantiate a logistic regression model with the **saga** solver, L1 penalty, and set **max_iter** to **1000** as we want the solver to have enough iterations to find a good solution.
5. Import the **Pipeline** class and create a pipeline with the scaler and the logistic regression model, using the names '**scaler**' and '**model**' for the steps, respectively.
6. Use the **get_params** and **set_params** methods to see how to view the parameters from each stage of the pipeline and change them.
7. Create a smaller range of C values to test with cross-validation, as these models will take longer to train and test with more data than our previous exercise; we recommend $C = [10^2, 10, 1, 10^{-1}, 10^{-2}, 10^{-3}]$.
8. Make a new version of the **cross_val_C_search** function called **cross_val_C_search_pipe**. Instead of the **model** argument, this function will take a **pipeline** argument. The changes inside the function will be to set the C value using **set_params(model__C = <value you want to test>)** on the pipeline, replacing the model with the pipeline for the **fit** and **predict_proba** methods, and accessing the C value using **pipeline.get_params()['model__C']** for the printed status update.
9. Run this function as in the previous exercise, but using the new range of C values, the pipeline you created, and the features and response variable from the training split of the case study data.

You may see warnings here, or in later steps, regarding the non-convergence of the solver; you could experiment with the `tol` or `max_iter` options to try and achieve convergence, although the results you obtain with `max_iter = 1000` are likely to be sufficient.

10. Plot the average training and test ROC AUC across folds for each C value.
11. Create interaction features for the case study data and confirm that the number of new features makes sense.
12. Repeat the cross-validation procedure and observe the model performance when using interaction features.

Note that this will take substantially more time, due to the larger number of features, but it will probably take less than 10 minutes. So, does the average cross-validation test performance improve with the interaction features? Is regularization useful?

Note

The Jupyter notebook containing the Python code for this activity can be found at <https://packt.link/ohGgX>. Detailed step-wise solution to this activity can be found via [this link](#).

Summary

In this chapter, we introduced the final details of logistic regression and continued to understand how to use scikit-learn to fit logistic regression models. We gained more visibility into how the model fitting process works by learning about the concept of a cost function, which is minimized by the gradient descent procedure to estimate parameters during model fitting.

We also learned of the need for regularization by introducing the concepts of underfitting and overfitting. In order to reduce overfitting, we saw how to adjust the cost function to regularize the coefficients of a logistic regression model using an L1 or L2 penalty. We used cross-validation to select the amount of regularization by tuning the regularization hyperparameter. To reduce underfitting, we saw how to do some simple feature engineering with interaction features for the case study data.

We are now familiar with some of the most important concepts in machine learning. We have, so far, only used a very basic classification model: logistic regression. However, as you increase your toolbox of models that you know how to use, you will find that the concepts of overfitting and underfitting, the bias-variance trade-off, and hyperparameter tuning will come up again and again. These ideas, as well as convenient scikit-learn implementations of the cross-validation functions that we wrote in this chapter, will help us through our exploration of more advanced prediction methods.

In the next chapter, we will learn about decision trees, an entirely different type of predictive model than logistic regression, and the random forests that are based on them. However, we will use the same concepts that we learned here, cross-validation and hyperparameter search, to tune these models.